

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# SE 423: Introduction to Mechatronics

## Lecture 6-7: Encoders & PWM

Marius Juston

Monday, February 9th, 2026

# Reminders

**Office Hours:**

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

**Lab:** Starting Lab 3 this week. This a 2-week lab!

**Homeworks:**

- Check-offs for homework 2 Ex 1 & 2 are due **February 17, 5PM (The end of office hours)**
- Check-offs for homework 2 Ex 3 & 4 are due **February 24, 5PM (The end of office hours)**
- Answers and code submissions for homework 2 are due **February 17, 9AM**

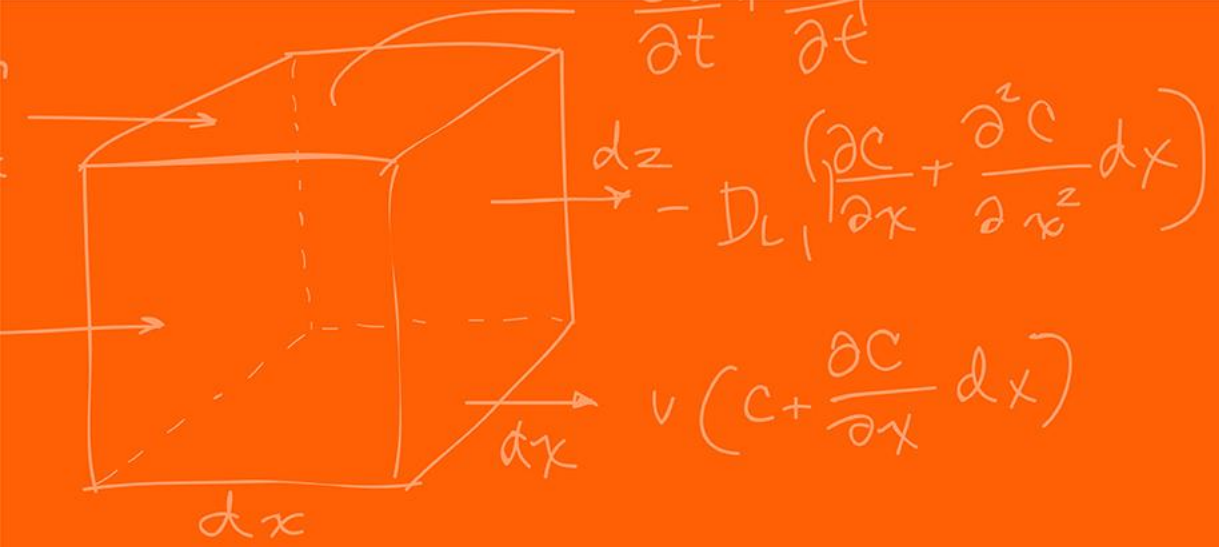
**LabVIEW:**

- All check-offs for LabVIEW 2 are due **February 26, 5PM**

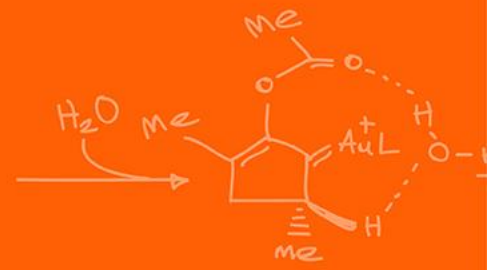
# Questions? (1/2)

Homework, Lab, LabVIEW questions?

(All submission times / days are final! Be early, as you see these HW can be very long! The times / days are always on Canvas / Gradescope / Lectures)

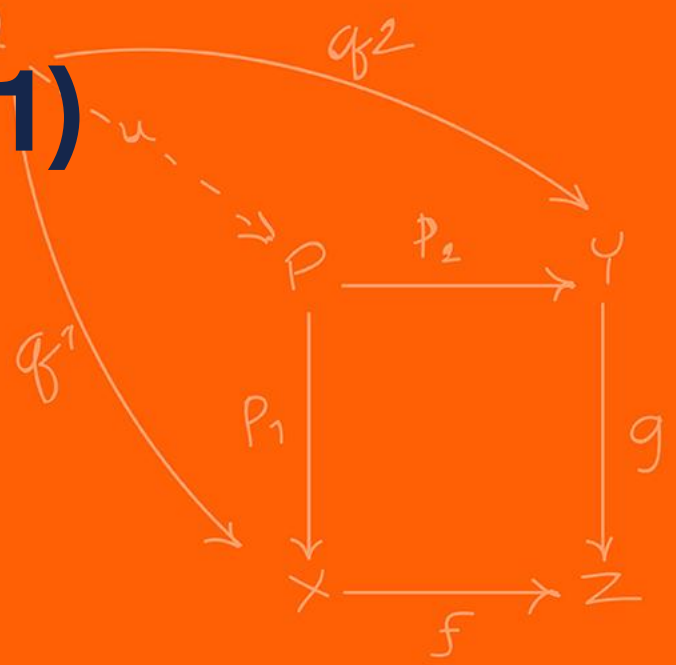








# Optical Encoder (1/11)



# Optical Encoder (2/11)

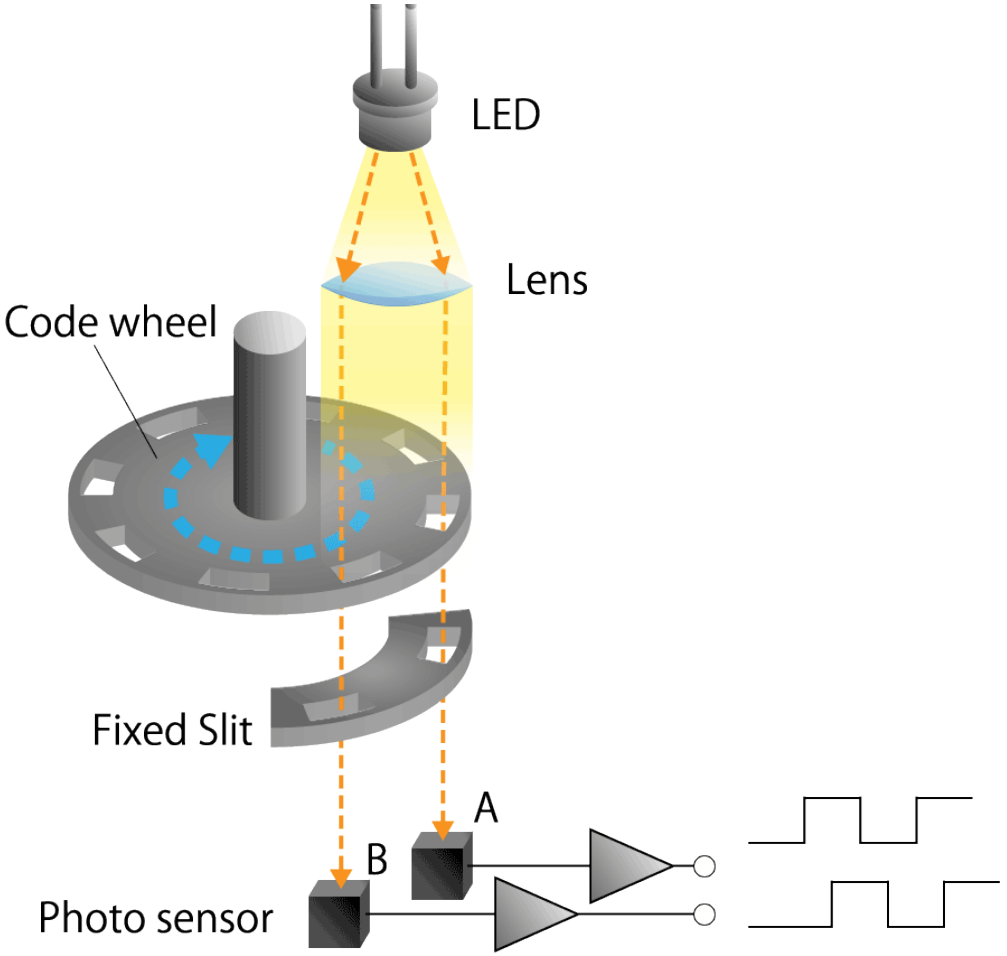
How do we measure angular position or motion of a shaft into a digital value? Ideas?

The optical encoder is composed of a light emitting device (**LED**), **photo sensors**, and a disc called a **code wheel** with slits (holes) in the radial direction and detects rotational position information as an optical pulse signal.

When a code wheel attached to a rotating shaft such as a motor rotates, an optical pulse is generated depending on whether light emitted from a fixed light emitting element passes through a slit of the code wheel or not.

Photo sensor detects the optical pulse, converts it into an electrical signal, and outputs it.

# Optical Encoder (3/11)




https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/optical-encoder/

# Optical Encoder (4/11)

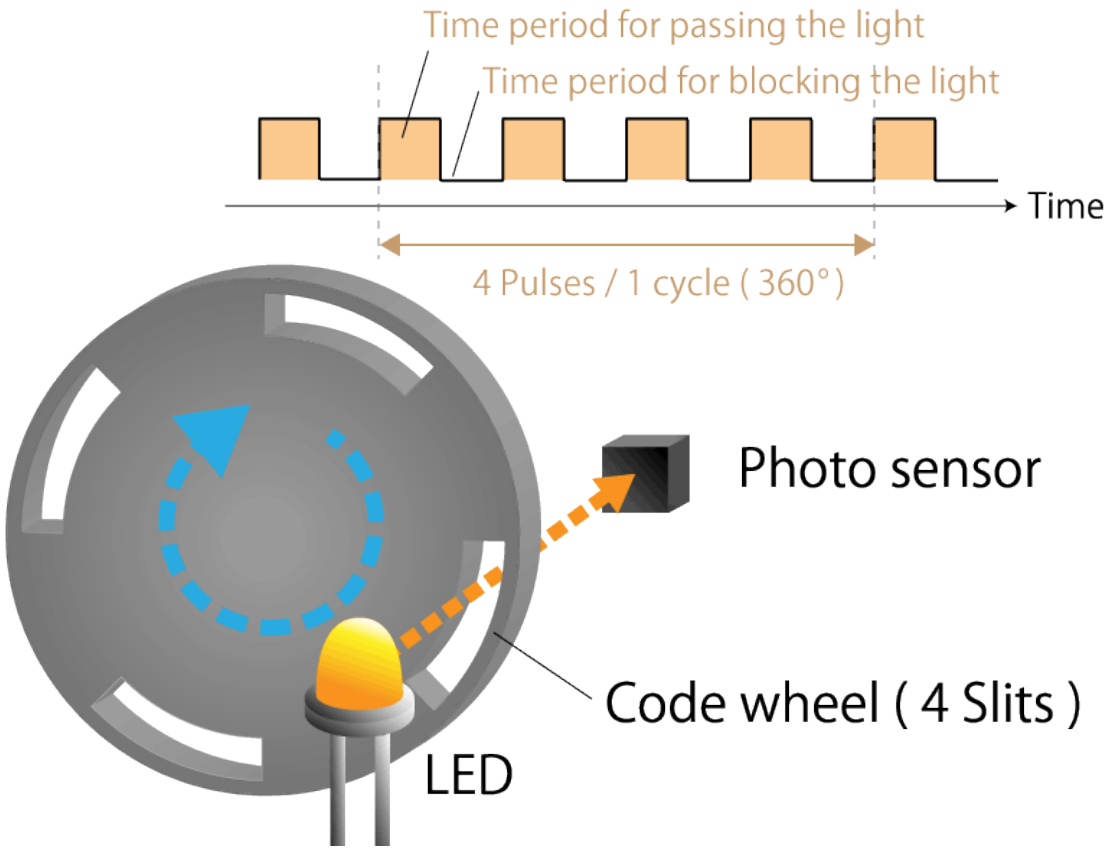
2 Different types of optical encoders,

An **incremental** optical encoder:

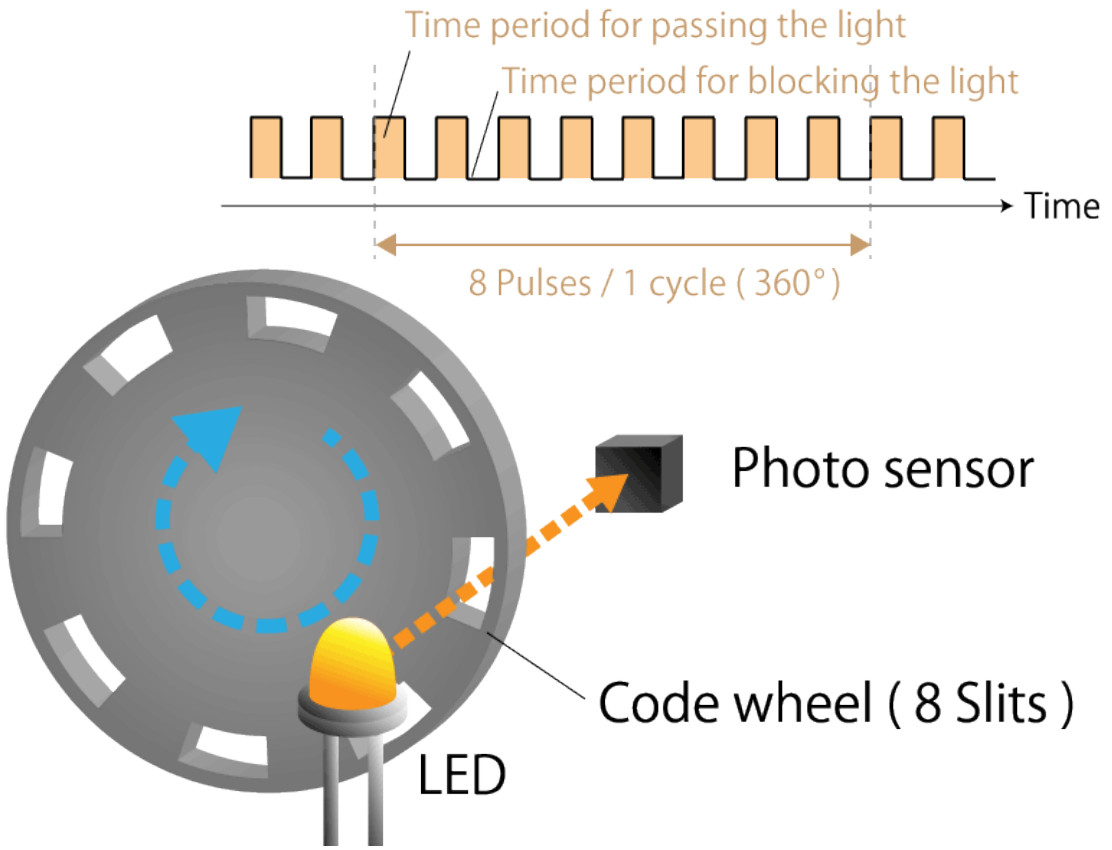
- The code disc divided into sectors that are alternately transparent and opaque.
- A light source is positioned on one side of the disc, and a light sensor on the other side.
- As the disc rotates, the output from the detector switches alternately on and off, depending on whether the sector appearing between the light source and the detector is transparent or opaque.

The encoder produces a stream of square wave pulses which, when counted, indicate the angular position of the shaft.

# Optical Encoder - Resolution




4-slit incremental type rotary encoder

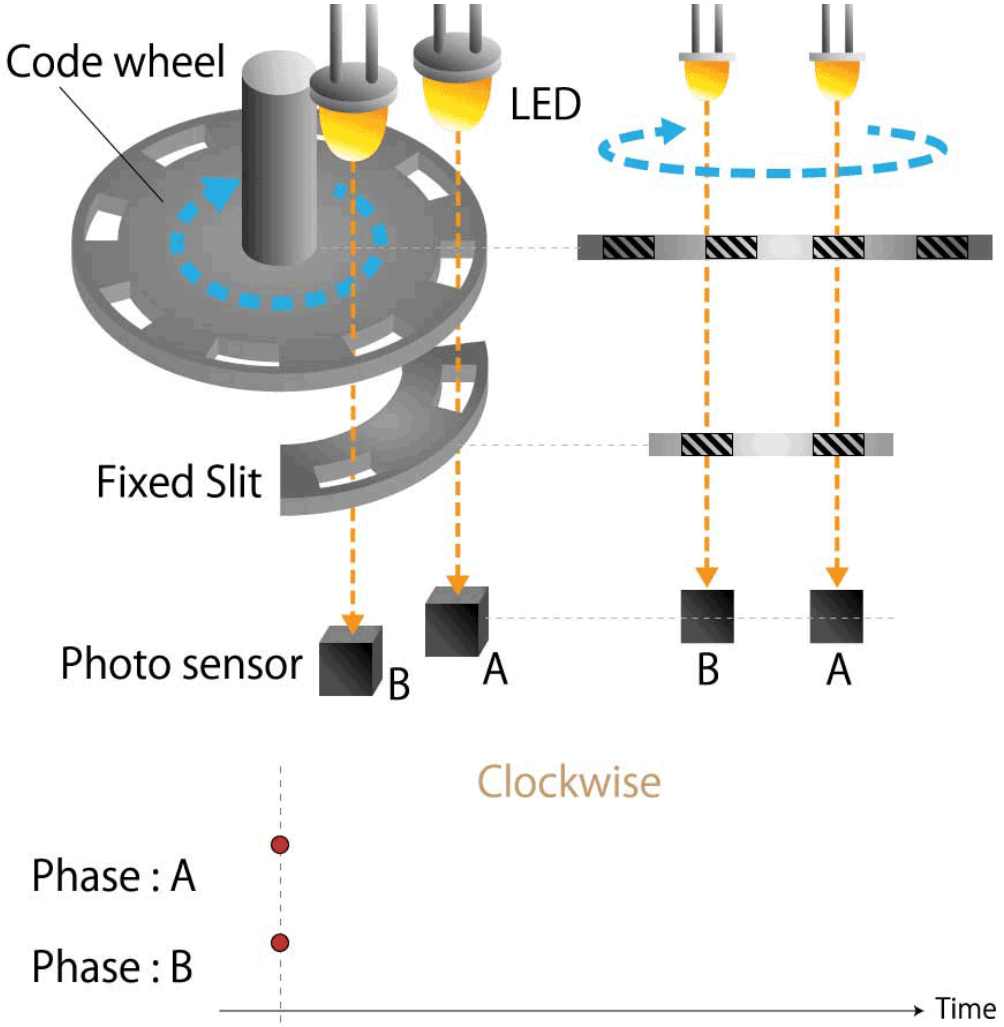



8-slit incremental type rotary encoder

https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

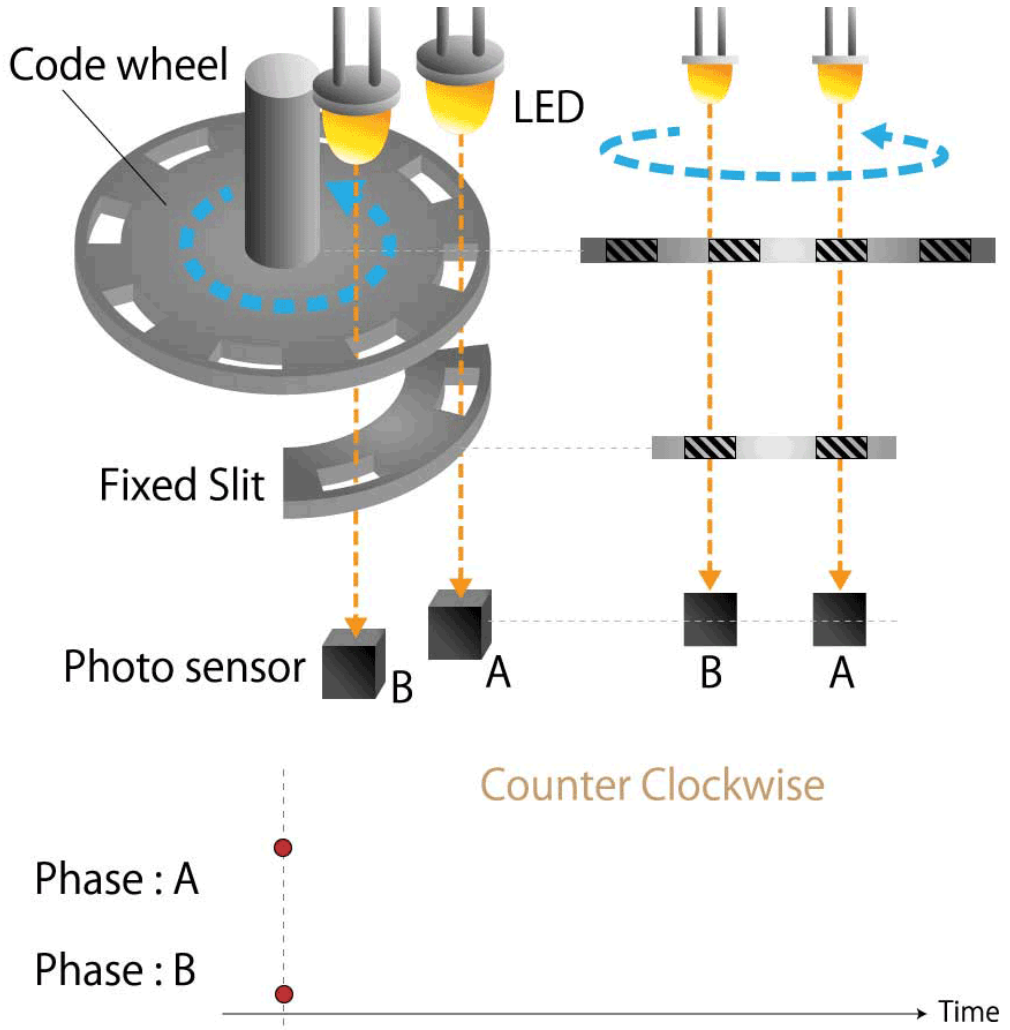
# What are some problems?

# Optical Encoder - Direction (1/6)



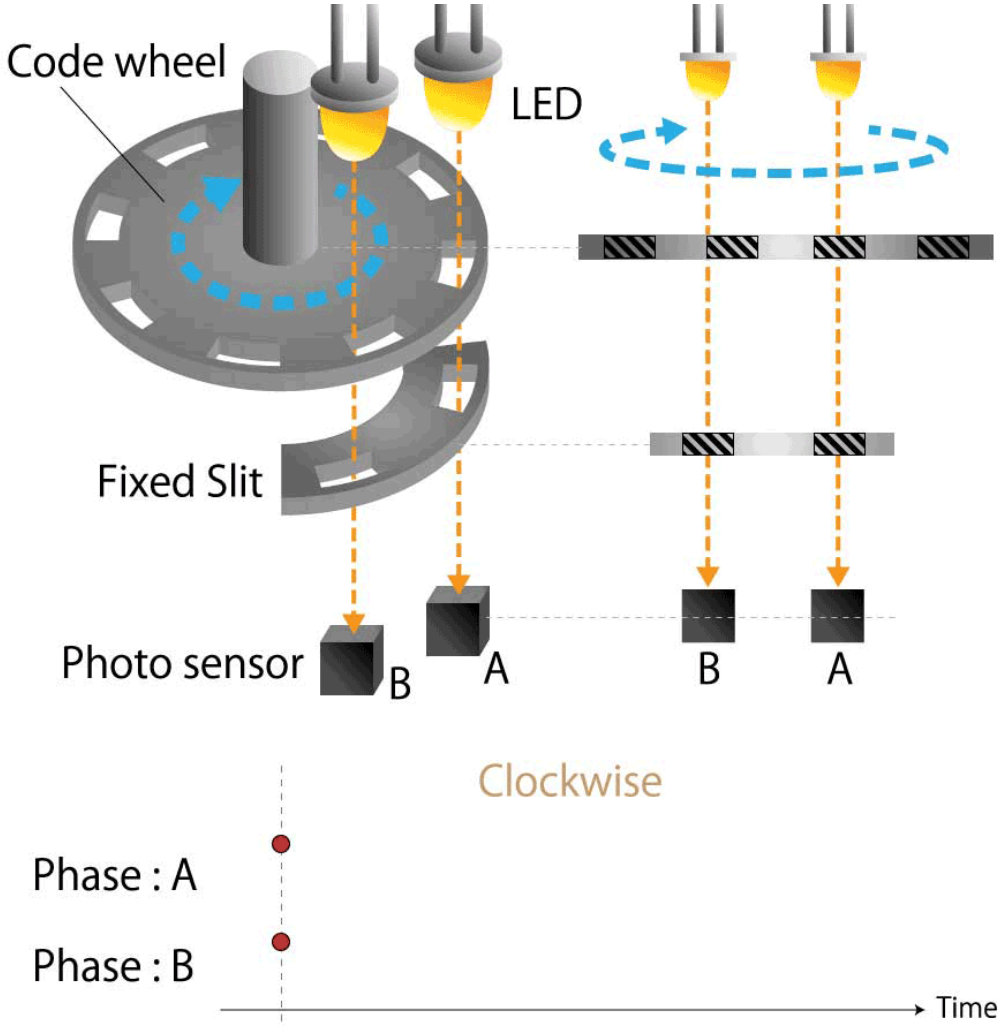

https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

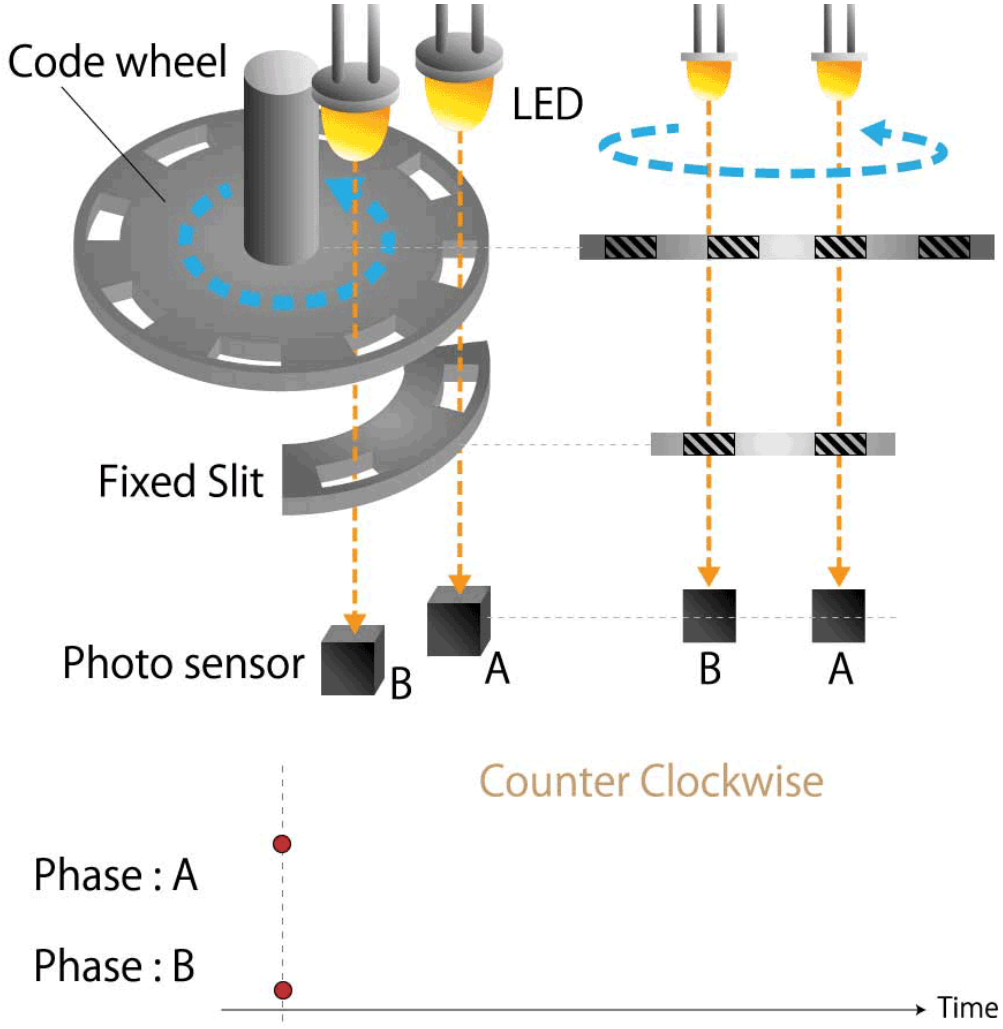
# Optical Encoder - Direction (2/6)




https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

# Optical Encoder - Direction (3/6)

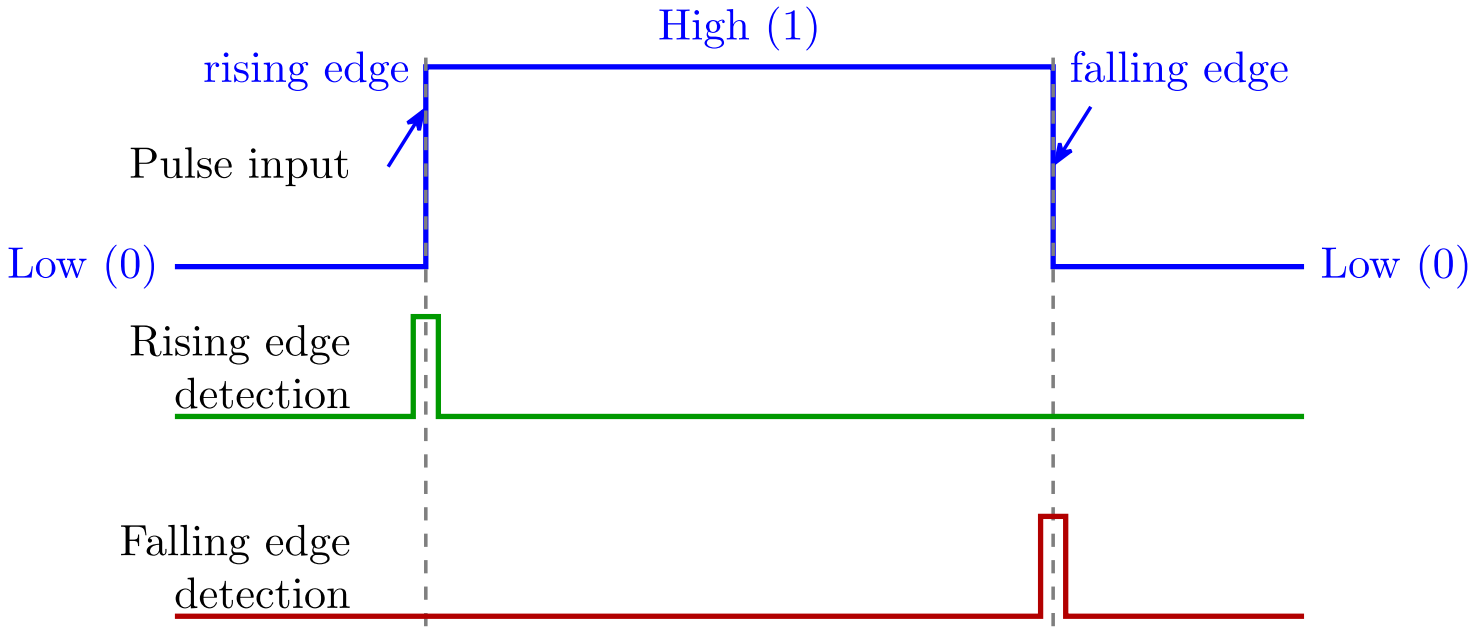



https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

# Optical Encoder - Direction (4/6)

- To encode the direction you have 2 channels, each of them offset by a phase of 90 degrees
- The rotational direction can be determined depending on which pulse of the phase A or phase B rises B.




- By subtracting the number of pulses in reverse rotation, the amount of rotation can be accurately determined.

# Optical Encoder - Direction (5/6)

- The combined output of (A, B), forms a 2-bit **Gray code sequence** (only 1 bit changes each timestep).
- The states cycle through 4 quadrants (00, 01, 10, 11)

If **clockwise** (CW): A leads B, the sequence is:

$$(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (0,1) \rightarrow (0,0)$$

If **counter-clockwise** (CCW): B leads C, the sequence is:

$$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (1,0) \rightarrow (0,0)$$

If I do not have an animation, I did not have time to make it.

# Optical Encoder - Direction (6/6)

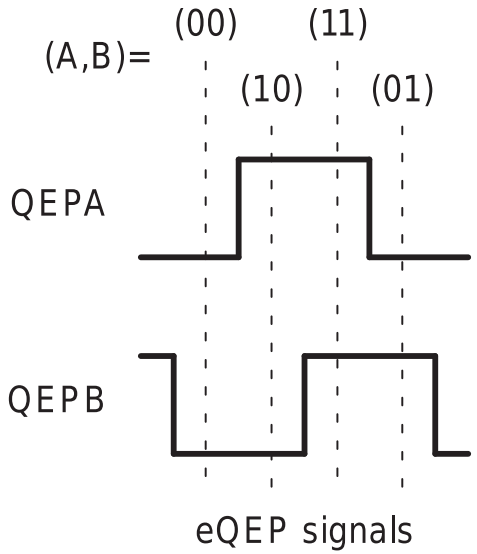
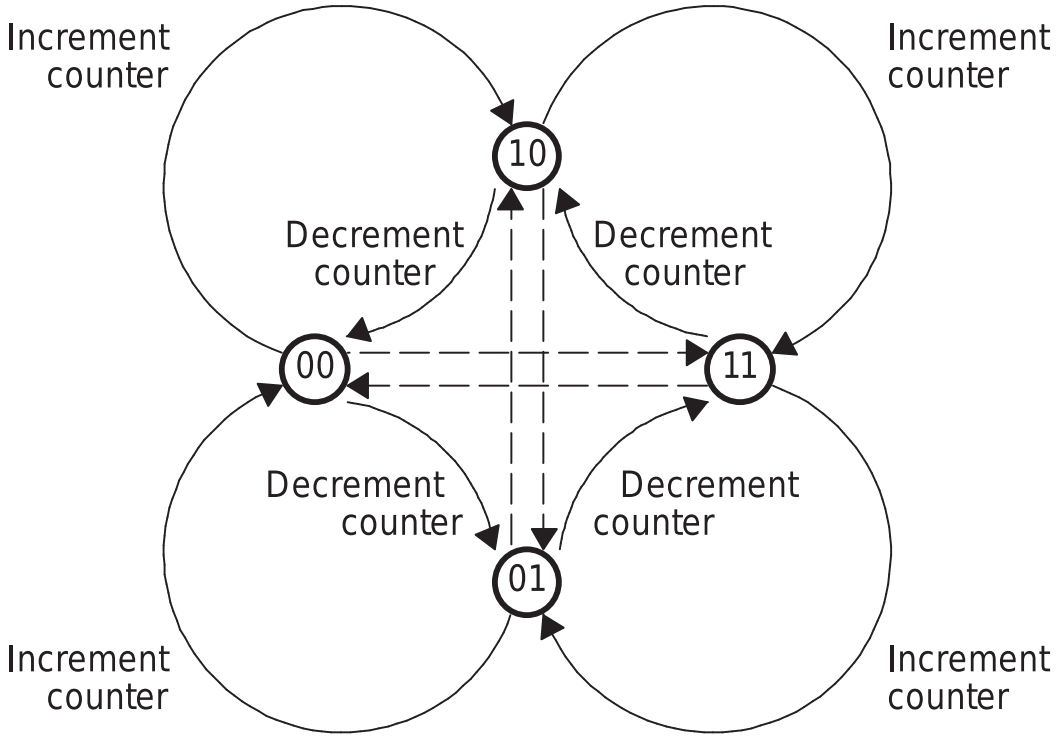
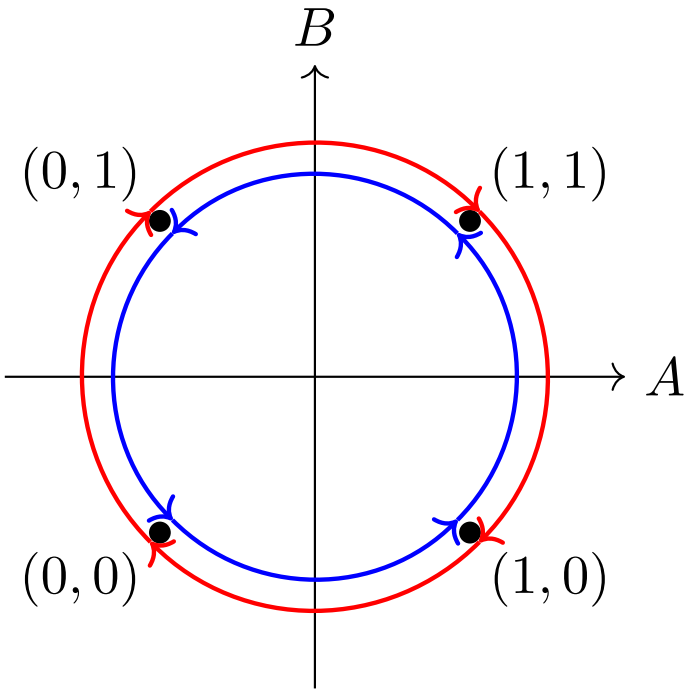



**Figure 17-6. Quadrature Decoder State Machine**



# What if you turn the robot off?

# Optical Encoder (5/11)

An **absolute** optical encoder

- An absolute optical encoder's disc is divided up into N sectors
  - Each sector is further divided radially along its length into opaque and transparent sections, forming a unique N-bit digital word with a maximum count of 2N – 1.
- The digital word formed radially by each sector increments in value from one sector to the next, usually employing **Gray code**.
- **Binary coding** could be used but can produce large errors if a single bit is incorrectly interpreted by the sensors.
- Gray code overcomes this defect: the maximum error produced by an error in any single bit of the Gray code is only 1 LSB after the Gray code is converted into binary code.
- A set of N light sensors responds to the **N-bit digital word** which corresponds to the **disc's absolute angular position**.

# Optical Encoder (6/11)

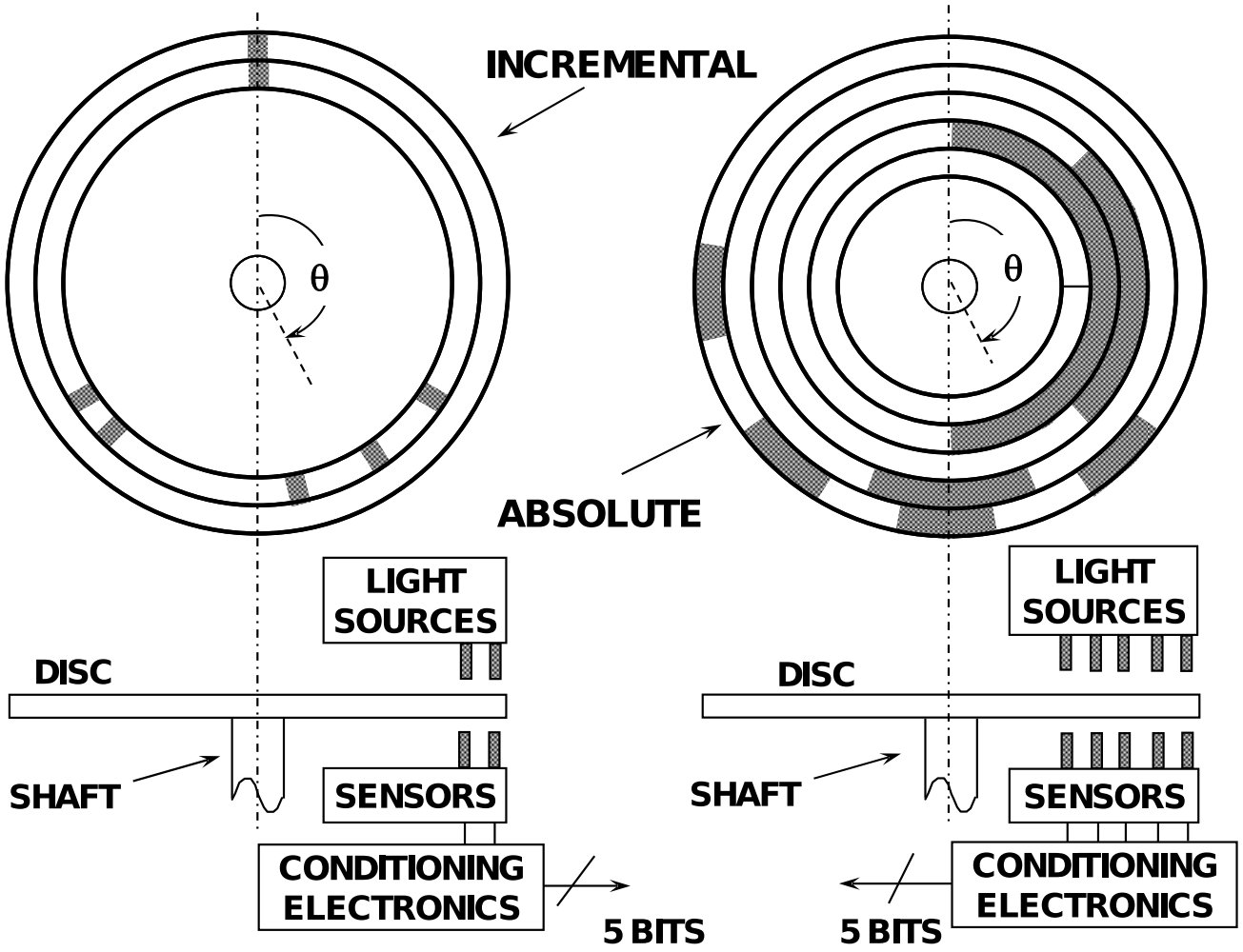
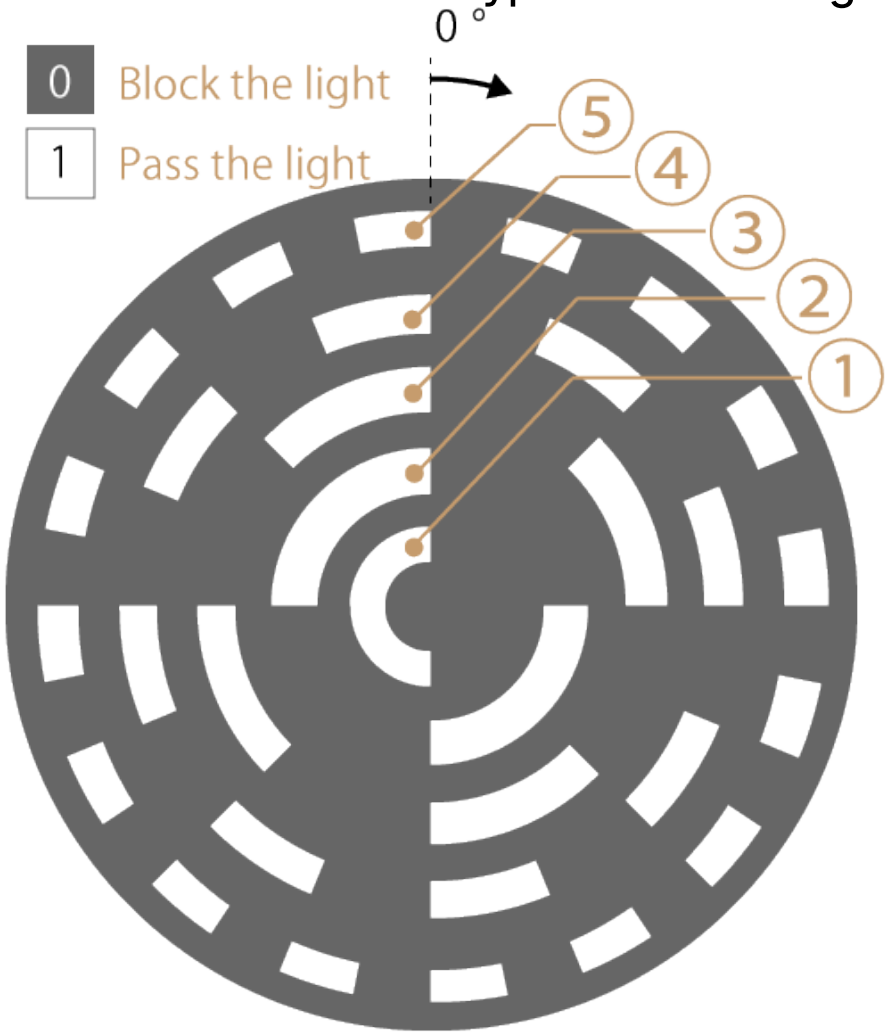



**Figure 1: Incremental and Absolute Optical Encoders**

https://www.analog.com/media/en/training-seminars/tutorials/mt-029.pdf

# Optical Encoder (7/11)

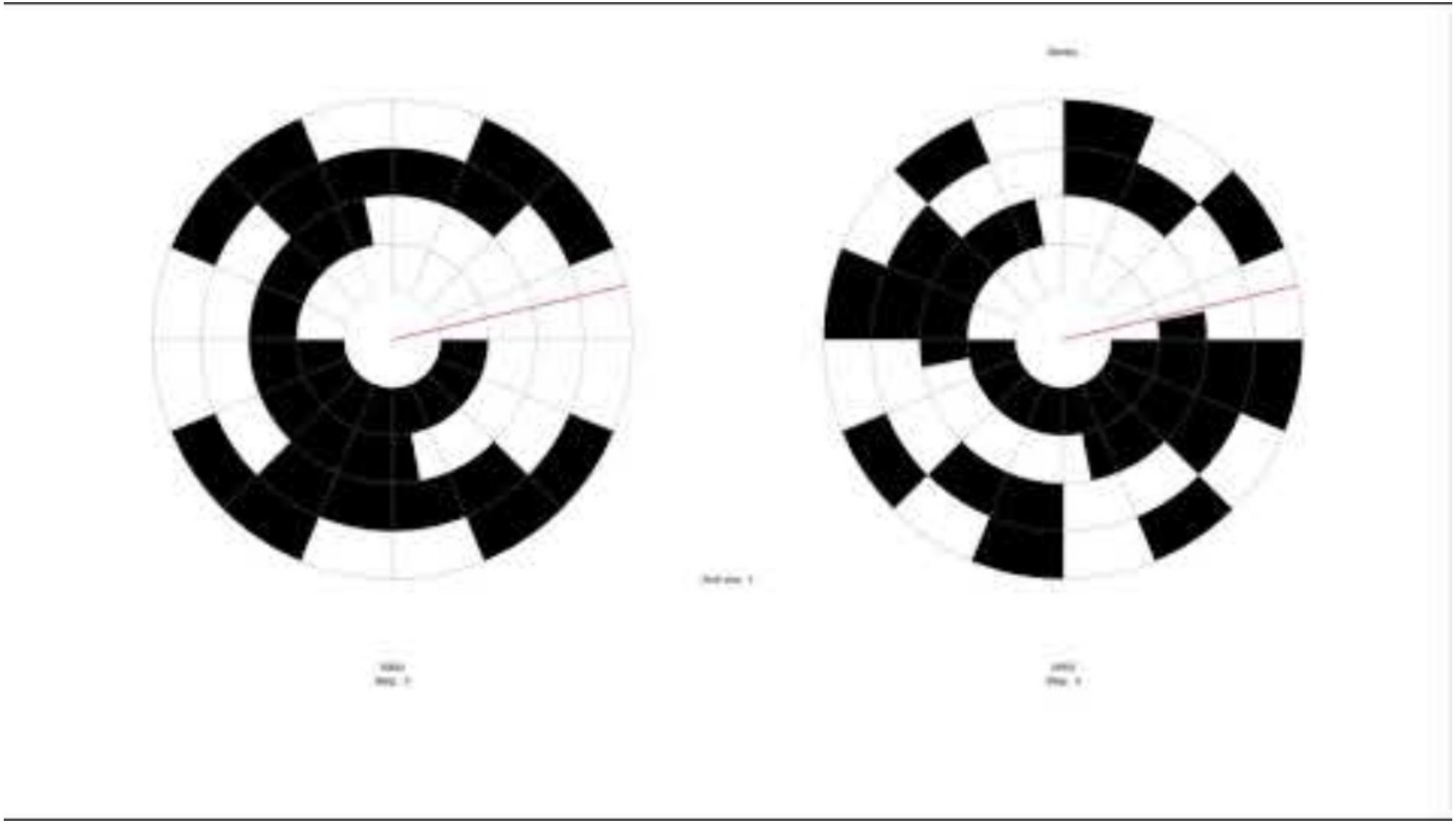
What type of encoding is this? Gray or binary?




| Angle [degrees] | ① | ② | ③ | ④ | ⑤ |
|---|---|---|---|---|---|
| 0.00 | 0 | 0 | 0 | 0 | 0 |
| 11.25 | 0 | 0 | 0 | 0 | 1 |
| 22.50 | 0 | 0 | 0 | 1 | 0 |
| 33.75 | 0 | 0 | 0 | 1 | 1 |
| 45.00 | 0 | 0 | 1 | 0 | 0 |
| 56.25 | 0 | 0 | 1 | 0 | 1 |
| 67.50 | 0 | 0 | 1 | 1 | 0 |
| 78.75 | 0 | 0 | 1 | 1 | 1 |
| 90.00 | 0 | 1 | 0 | 0 | 0 |
| . | . | . | . | . | . |
| . | . | . | . | . | . |
| . | . | . | . | . | . |
| 315.00 | 1 | 1 | 1 | 0 | 0 |
| 326.25 | 1 | 1 | 1 | 0 | 1 |
| 337.50 | 1 | 1 | 1 | 1 | 0 |
| 348.75 | 1 | 1 | 1 | 1 | 1 |

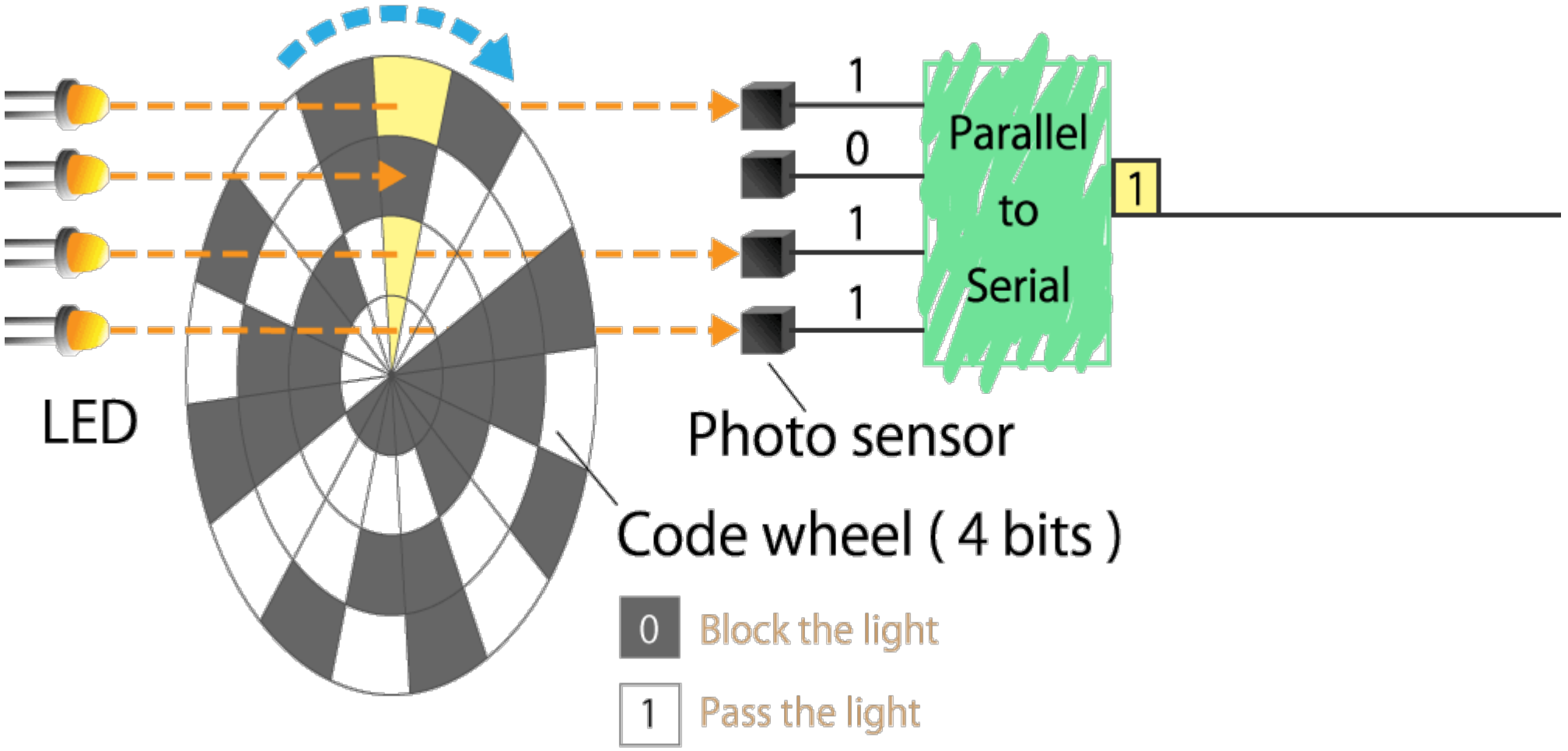
https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

# Optical Encoder (8/11)

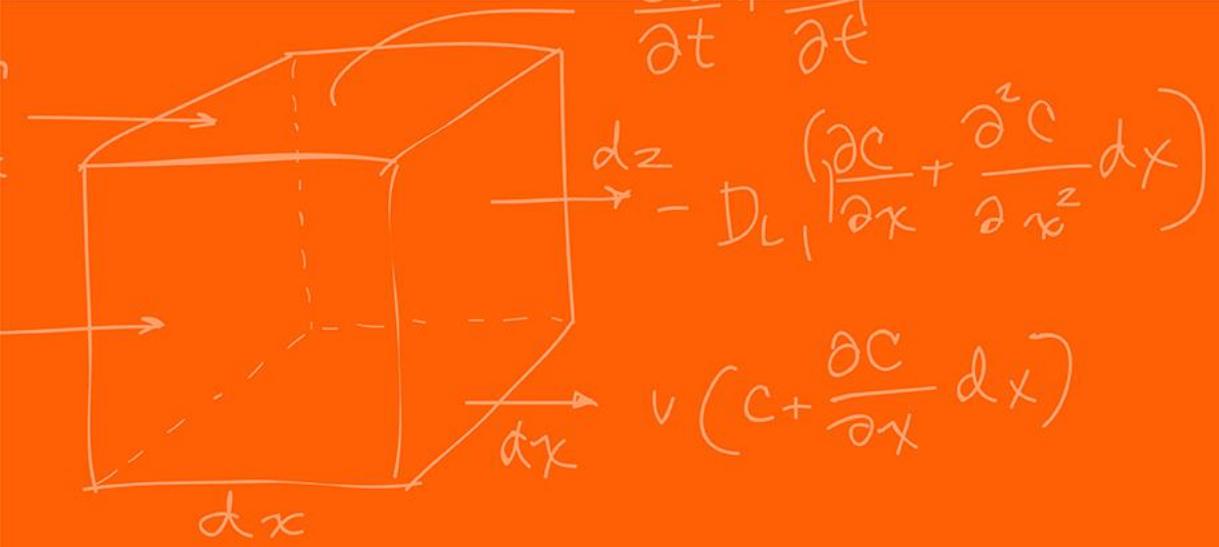


https://youtu.be/VKmMoT5hpVY

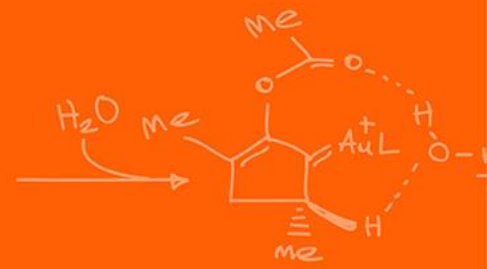
# Optical Encoder (9/11)




https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/

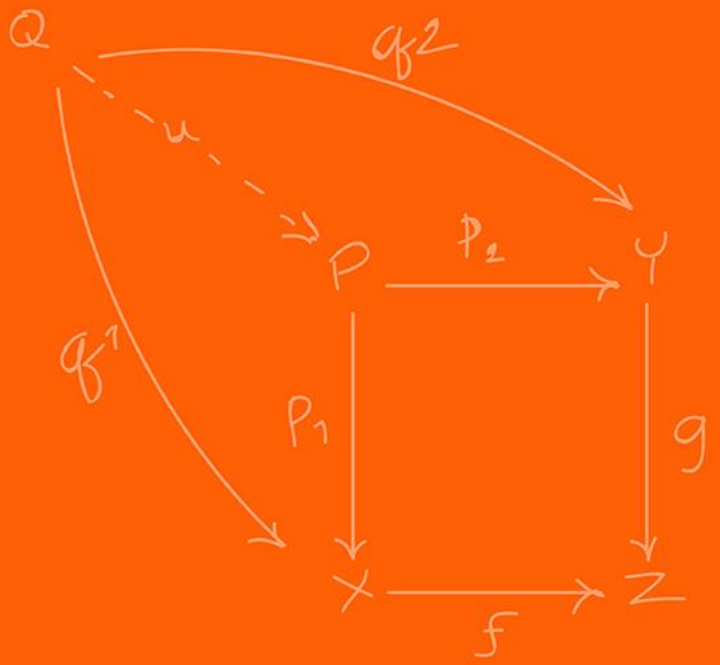








# eQEP (1/7)



# eQEP (2/7)

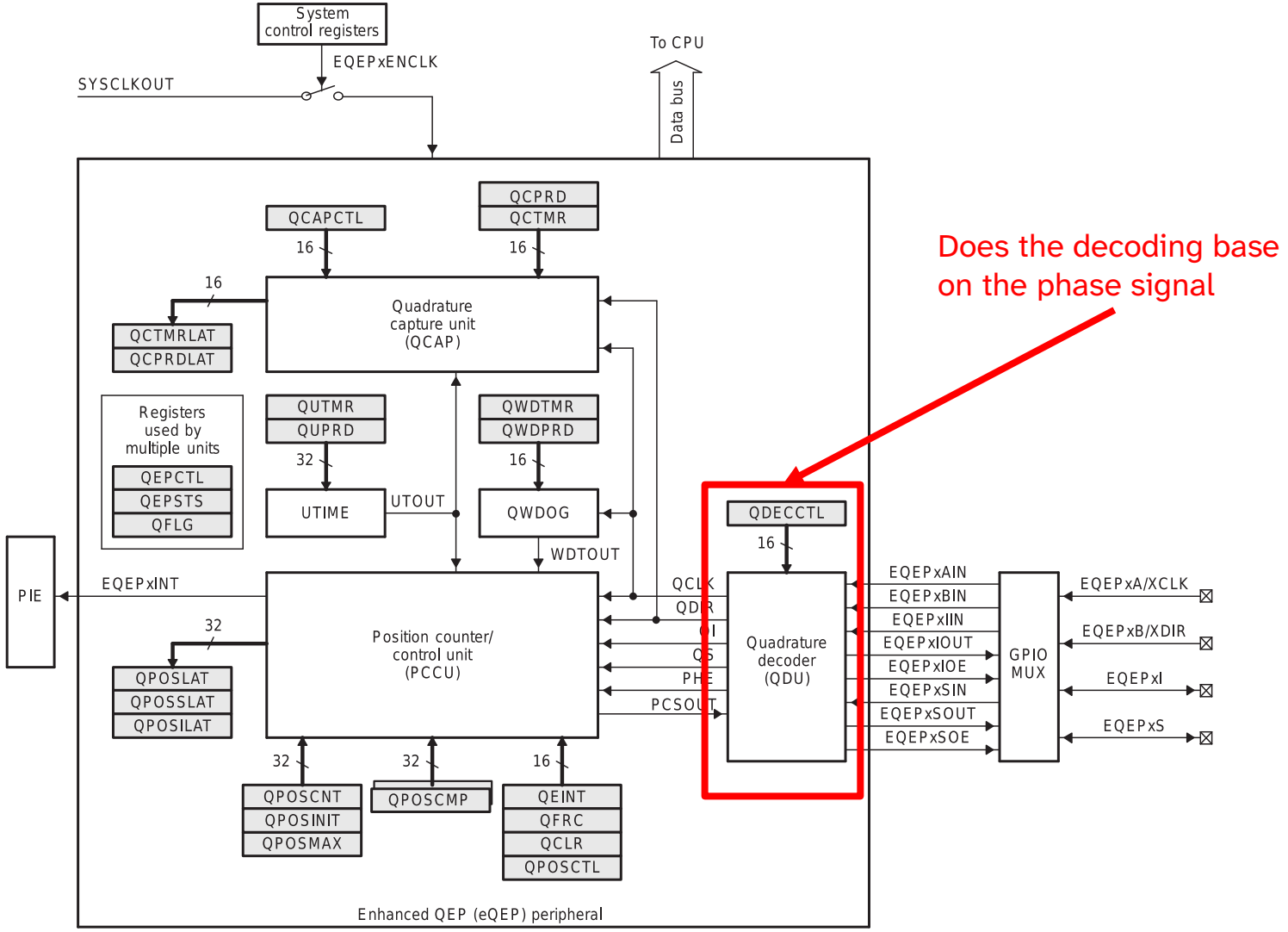
The eQEP peripheral is used to read data from an **incremental** optical encoder only.

As such what is the main data that needs to be read in?

The 2 phase signals!

# eQEP – Overview diagram

**Figure 17-4. Functional Block Diagram of the eQEP Peripheral**



# eQEP (3/7)

```
// setup eQEP1 pins for input
EALLOW;
//Disable internal pull-up for the selected output pins for reduced power
consumption
GpioCtrlRegs.GPAPUD.bit.GPIO20 = 1;    // Disable pull-up on GPIO20 (EQEP1A)
GpioCtrlRegs.GPAPUD.bit.GPIO21 = 1;    // Disable pull-up on GPIO21 (EQEP1B)
GpioCtrlRegs.GPAQSEL2.bit.GPIO20 = 2;   // Qual every 6 samples
GpioCtrlRegs.GPAQSEL2.bit.GPIO21 = 2;   // Qual every 6 samples
EDIS;
```

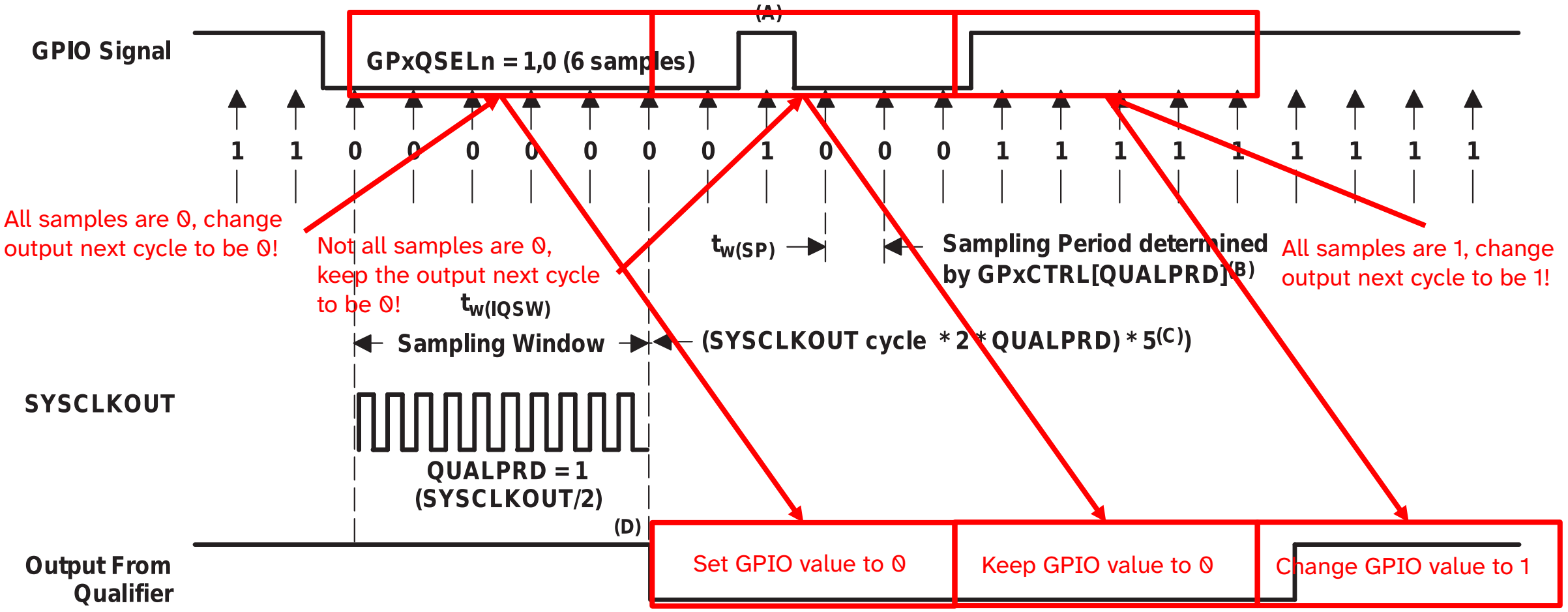
# eQEP - QPxQSELn (1/2)

The number of times the signal is sampled is either three samples or six samples as specified in the qualification selection (GPAQSEL1, GPAQSEL2, GPBQSEL1, and GPBQSEL2) registers. When three or six consecutive cycles are the same, then the input change will be passed through to the MCU.

All the samples must be **consistent within that sampling period** to change its output for the next output range.

Helps with reducing noise as a random change would not change the result.

# eQEP - QPxQSELn (2/2)



# eQEP (4/7)

```
// setup eQEP1 pins for input
// This specifies which of the possible GPIO pins will be EQEP1 functional pins.
GPIO_SetupPinMux(20, GPIO_MUX_CPU1, 1); // set pin mux EQEP2A
GPIO_SetupPinMux(21, GPIO_MUX_CPU1, 1); // set pin mux EQEP2B
EQep1Regs.QEPCTL.bit.QPEN = 0;          // make sure eQEP in reset
EQep1Regs.QDECCTL.bit.QSRC = 0;         // Quadrature count mode
EQep1Regs.QPOSCTL.all = 0x0;            // Disable eQEP Position Compare
EQep1Regs.QCAPCTL.all = 0x0;            // Disable eQEP Capture
EQep1Regs.QEINT.all = 0x0;              // Disable all eQEP interrupts
EQep1Regs.QPOSMAX = 0xFFFFFFFF;         // use full range of the 32-bit count
EQep1Regs.QEPCTL.bit.FREE_SOFT = 2;     // eQEP unaffected by emulation suspend in Code
Composer
EQep1Regs.QPOSCNT = 0;                  // Set the initial position of the encoder
EQep1Regs.QEPCTL.bit.QPEN = 1;          // Enable eQEP
```

# eQEP (5/7)

GPIO Mux Selection

| | | GPIO Index | 0,4,8,12 | 1 | 2 | 3 | 5 | 6 | 7 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|
| ME461 | LaunchPad | GPyGMUXn | 00,01,10,11 | 00b | | | 01b | | | 11b |
| Default | Pins | GPyMUXn | 00b | 01b | 10b | 11b | 01b | 10b | 11b | 11b |
| WIZNET RST | 40/J4.10 | | GPIO0 | EPWM1A | | | | SDAA | | |
| ESP8266 RST | 39/J4.9 | | GPIO1 | EPWM1B | | MFSRB | | SCLA | | |
| 4793PWM1 | 38/J4.8 | | GPIO2 | EPWM2A | | | OUTXBAR1 | SDAB | | |
| 4793PWM2 | 37/J4.7 | | GPIO3 | EPWM2B | OUTXBAR2 | MCLKRB | OUTXBAR2 | SCLB | | |
| Pushbotton1 | 36/J4.6 | | GPIO4 | EPWM3A | | | OUTXBAR3 | CANTXA | | |
| Pushbotton2 | 35/J4.5 | | GPIO5 | EPWM3B | MFSRA | OUTXBAR3 | | CANRXA | | |
| Pushbotton3 | 80/J8.10 | | GPIO6 | EPWM4A | OUTXBAR4 | EXTSYNCOUT | EQEP3A | CANTXB | | |
| Pushbotton4 | 79/J8.9 | | GPIO7 | EPWM4B | MCLKRA | OUTXBAR5 | EQEP3B | CANRXD | | |
| JoyPushBtn | 78/J8.8 | | GPIO8 | EPWM5A | CANTXB | ADCSOCAO | EQEP3S | SCITXDA | | |
| DAN28027 CS | 77/J8.7 | | GPIO9 | EPWM5B | SCITXDB | OUTXBAR6 | EQEP3I | SCIrXDA | | |
| DRV1PWM | 76/J8.6 | | GPIO10 | EPWM6A | CANRXB | ADCSOCBO | EQEP1A | SCITXDB | | UPP-WAIT |
| DRV2PWM | 75/J8.5 | | GPIO11 | EPWM6B | SCIRXDB | OUTXBAR7 | EQEP1B | SCIRXDB | | UPP-START |
| CANTXB | J12 | | GPIO12 | EPWM7A | CANTXB | MDXB | EQEP1S | SCITXDC | | UPP-ENA |
| RCServo1 | 74/J8.4 | | GPIO14 | EPWM8A | SCITXDB | MCLKXB | | OUTXBAR3 | | UPP-D6 |
| RCServo2 | 73/J8.3 | | GPIO15 | EPWM8B | SCIRXDB | MFSXB | | OUTXBAR4 | | UPP-D5 |
| BUZZER | 33/J4.3 | | GPIO16 | SPISIMOA | CANTXB | OUTXBAR7 | EPWM9A | | SD1_D1 | UPP-D4 |
| CANRXB | J12 | | GPIO17 | SPISOMIA | CANRXB | OUTXBAR8 | EPWM9B | | SD1_C1 | UPP-D3 |
| SPICLKA/SPIRAM | 4/J1.4 | | GPIO18 | SPICLKA | SCITXDB | CANRXA | EPWM10A | | SD1_D2 | UPP-D2 |
| SPIRAM CS | 3/J1.3 | | GPIO19 | SPISTEA | SCIRXDB | CANTXA | EPWM10B | | SD1_C2 | UPP-D1 |
| EQEP1A | J14 | | GPIO20 | EQEP1A | MDXA | CANTXb | EPWM11A | | SD1_D3 | UPP-D0 |
| EQEP1B | J14 | | GPIO21 | EQEP1B | MDRA | CANRXB | EPWM11B | | SD1_C3 | UPP-CLK |
| LED1/PWM | 8/J1.8 | | GPIO22 | EQEP1S | MCLKXA | SCITXDB | EPWM12A | SPICLKB | SD1_D4 | |
| WIZNET INT | 34/J4.4 | | GPIO24 | OUTXBAR1 | EQEP2A | MDXB | | SPISIMOB | SD2_D1 | |
| LED8 | 51/J6.1 | | GPIO25 | OUTXBAR2 | EQEP2B | MDRB | | SPISOMIB | SD2_C1 | |
| LED9 | 53/J6.3 | | GPIO26 | OUTXBAR3 | EQEP2I | MCLKXB | OUTXBAR3 | SPICLKB | SD2_D2 | |
| LED10 | 52/J6.2 | | GPIO27 | OUTXBAR4 | EQEP2S | MFSXB | OUTXBAR4 | SPISTEB | SD2_C2 | |
| DRV1DIR | 11/J2.1 | | GPIO29 | SCITXDA | EMSDCKE | | OUTXBAR6 | EQEP3B | SD2_C3 | |
| Board Blue LED | | | GPIO31 | CANTXA | EM1WE | | OUTXBAR8 | EQEP3I | SD2_C4 | |
| DRV2DIR | 2/J1.2 | | GPIO32 | SDAA | EM1CS0 | | | | | |
| Board Red LED | | | GPIO34 | OUTXBAR1 | EM1CS2 | | | SDAB | | |
| SDAB/BQ32000 | 50/J5.10 | | GPIO40 | | EM1A2 | | | SDAB | | |
| SCLB/BQ32000 | 49/J5.9 | | GPIO41 | | EM1A3 | | | SCLB | | |
| SCITXDA | FTDI | | GPIO42 | | | | | SDAA | | SCITXDA |
| SCIRXDA | FTDI | | GPIO43 | | | | | SCLA | | SCIRXDA |
| GPIO52 | 48/J5.8 | | GPIO52 | EQEP1S | EM1A12 | | | SPICLKC | SD1_D3 | |
| EQEP2A | J15 | | GPIO54 | SPISIMOA | EM1D30 | EM2D1 | EQEP2A | SCITXDB | SD1_D4 | |
| EQEP2B | J15 | | GPIO55 | SPISOMIA | EM1D29 | EM2D1 | EQEP2B | SCIRXDB | SD1_C4 | |

# eQEP – QDECCTL:QSRC

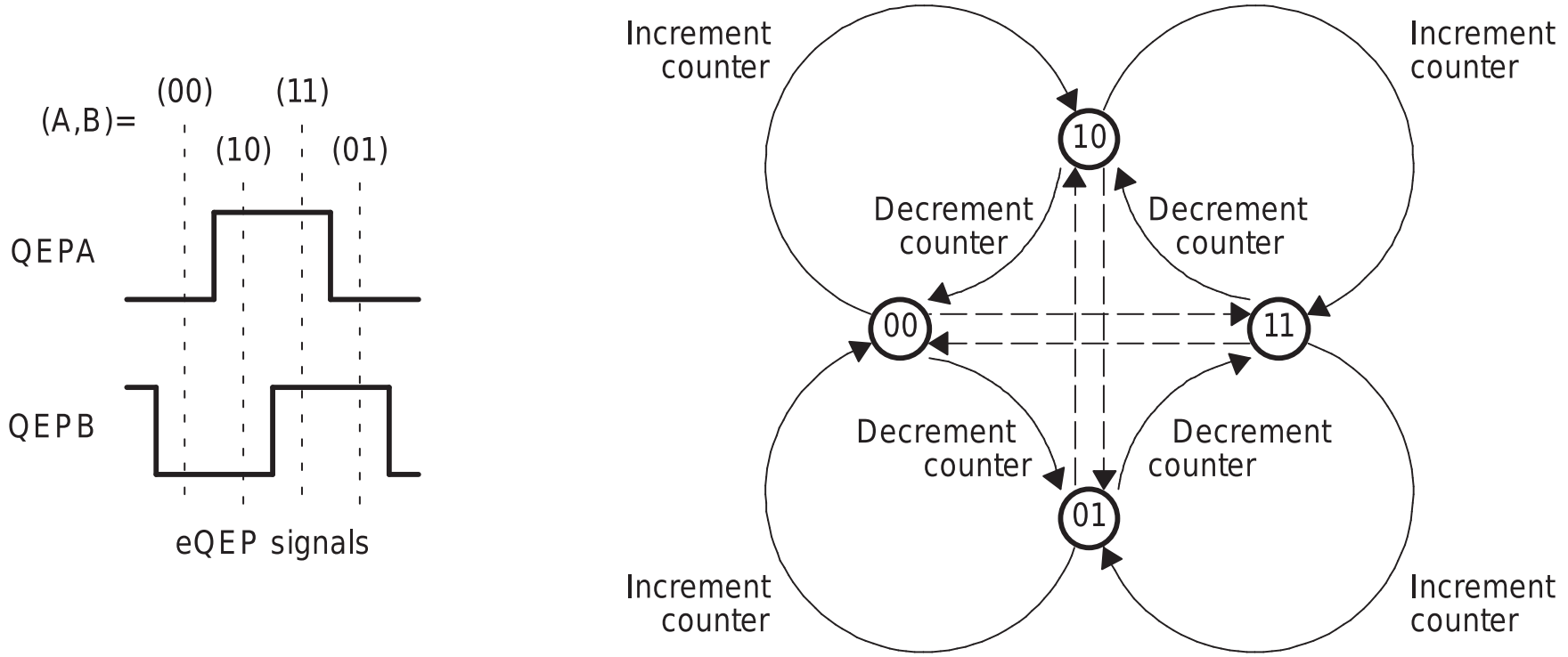
Position-counter source selection

- 0h (R/W) = Quadrature count mode (QCLK = iCLK, QDIR = iDIR)
- 1h (R/W) = Direction-count mode (QCLK = xCLK, QDIR = xDIR)
- 2h (R/W) = UP count mode for frequency measurement (QCLK = xCLK, QDIR = 1)
- 3h (R/W) = DOWN count mode for frequency measurement (QCLK= xCLK, QDIR = 0)

# eQEP – QDECCTL:QSRC – Quadrature Count Mode

The quadrature decoder generates the direction and clock to the position counter in quadrature count mode. Uses the 2-phase input to get the position and direction, this is the standard method.

**Figure 17-6. Quadrature Decoder State Machine**



# eQEP – QDECCTL:QSRC – Others Mode

**Direction-Count Mode**

Some position encoders provide direction and clock outputs, instead of quadrature outputs. In such cases, direction-count mode can be used.

- QEPA input will provide the clock for the position counter
- QEPB input will have the direction information.

The position counter is incremented on every rising edge of a QEPA input when the direction input is high, and decremented when the direction input is low.

**Up-Count Mode / Down-Count Mode**

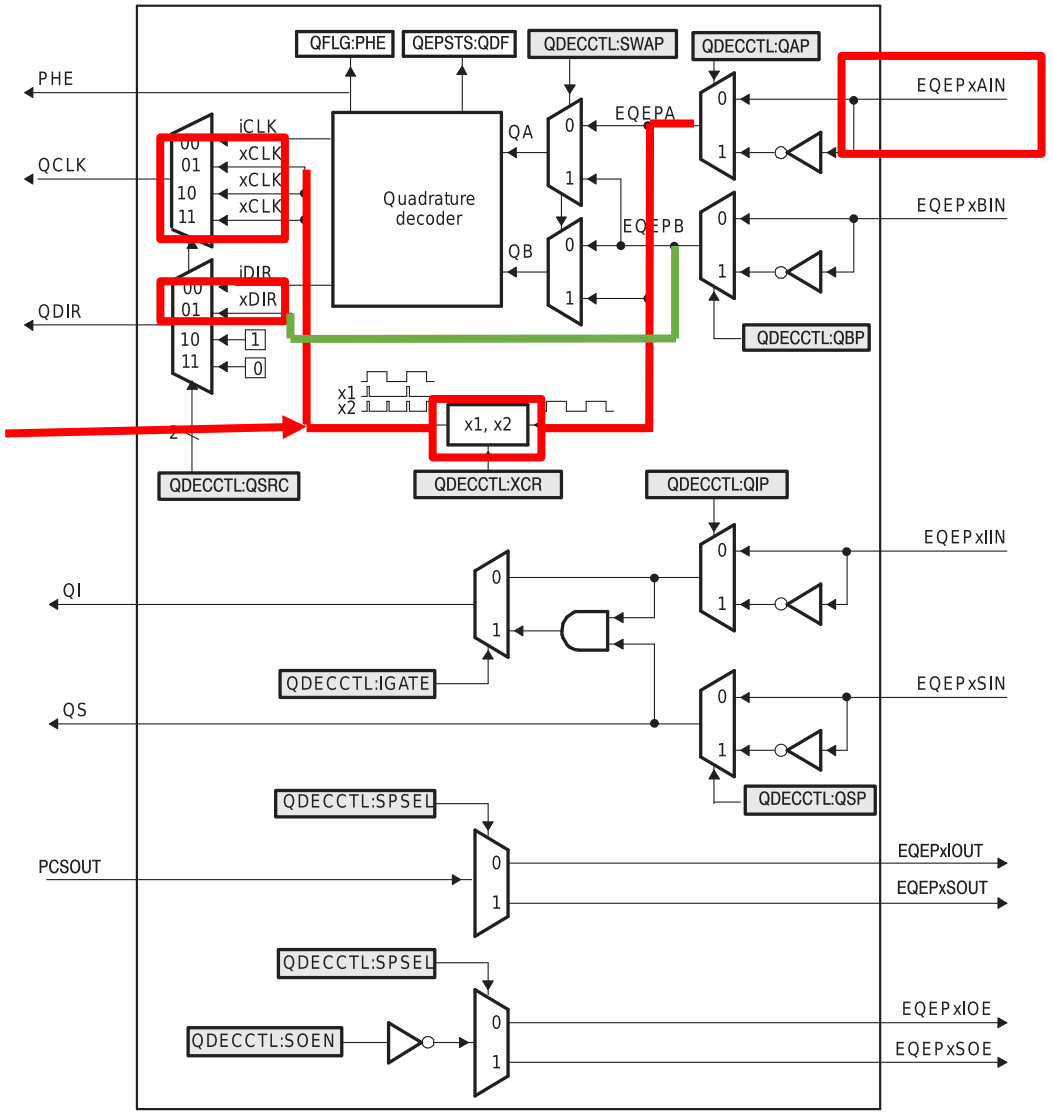
The counter direction signal is hard-wired for up / down -count and the position counter is used to measure the frequency of the QEPA input.

In up-count mode, it is recommended that the application not configure QEPB as a GPIO mux option, or ensure that a signal edge is not generated on the QEPB input.

# eQEP (6/7)

Direction count mode

**Figure 17-5. Functional Block Diagram of Decoder Unit**



# eQEP - CAP

The eQEP peripheral includes an integrated edge capture unit to measure the elapsed time between the unit position.

This feature is typically used for low-speed measurement using the following equation:

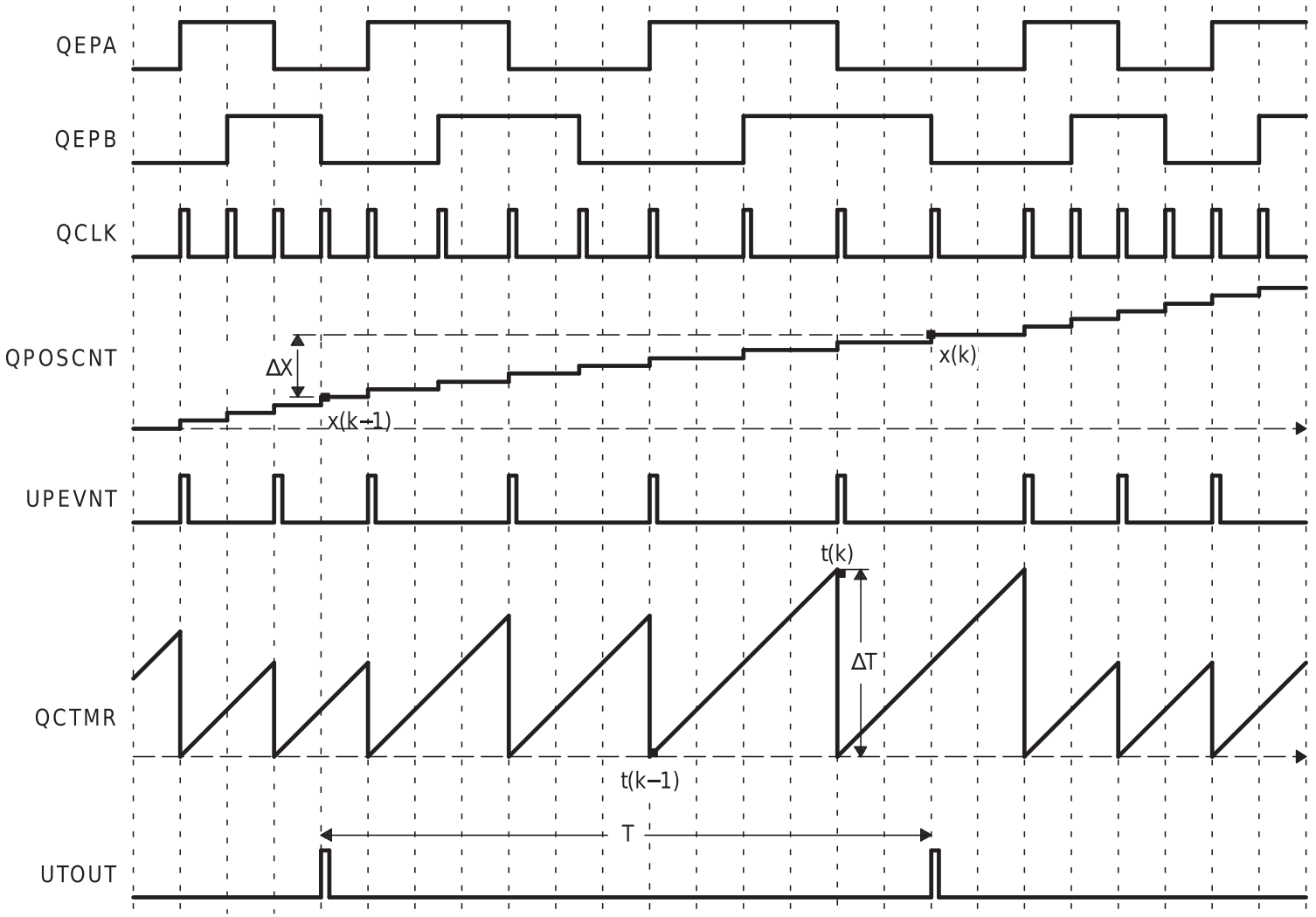
$$v(k) = \frac{X}{t(k) - t(k-1)} = \frac{X}{\Delta T}$$

Where,

- $X$: Unit position defined by integer multiple of quadrature edges
- $\Delta T$: Elapsed time between unit position events
- $v(k)$: Velocity at time instant "$k$"

# Optical Encoder (10/11)

**Figure 17-17. eQEP Edge Capture Unit - Timing Details**



# eQEP (7/7)

- The eQEP peripheral includes a 32-bit timer (QUTMR) that is clocked by SYSCLKOUT to generate periodic interrupts for velocity calculations. Whenever the unit timer (QUTMR) matches the unit period register (QUPRD), it resets the unit timer (QUPRD) it resets the unit timer (QUTMR).

Time measurement ($\Delta T$) between unit position events will be correct if the following conditions are met:

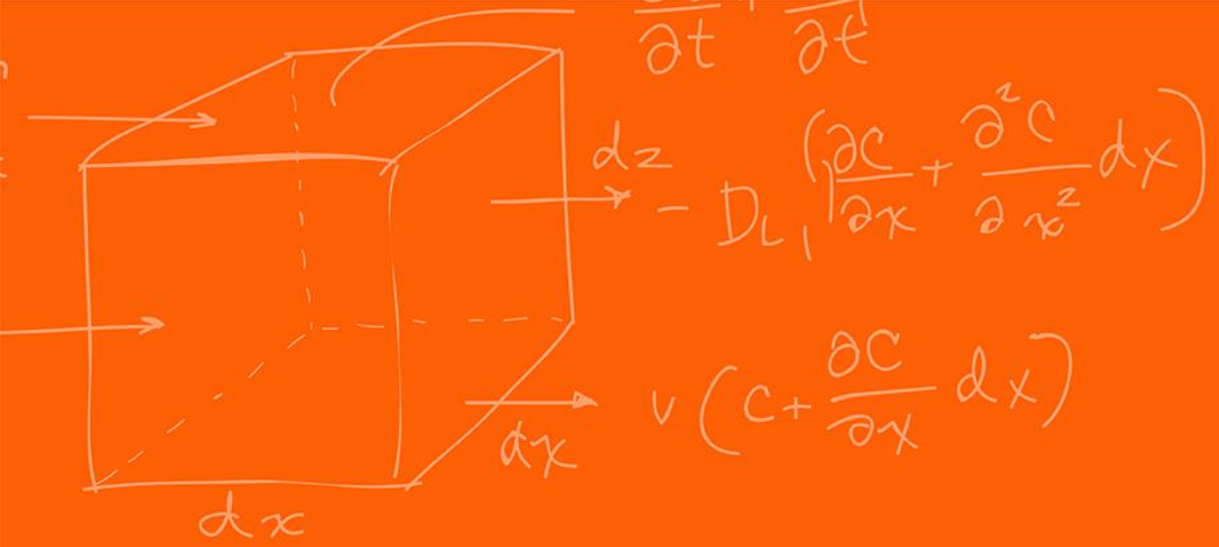
- No more than 65,535 counts have occurred between unit position events (16-bit register).
- No direction change between unit position events.

# Optical Encoder (11/11)

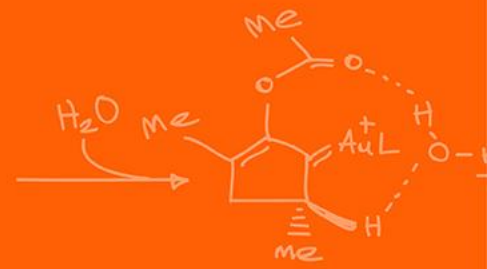
```
float readEncLeft(void) {
    int32_t raw = EQep1Regs.QPOSCNT;

    return (raw*(-2*PI/(2000.0*20.0)));
}
```

Simple as that! You just read the registry!

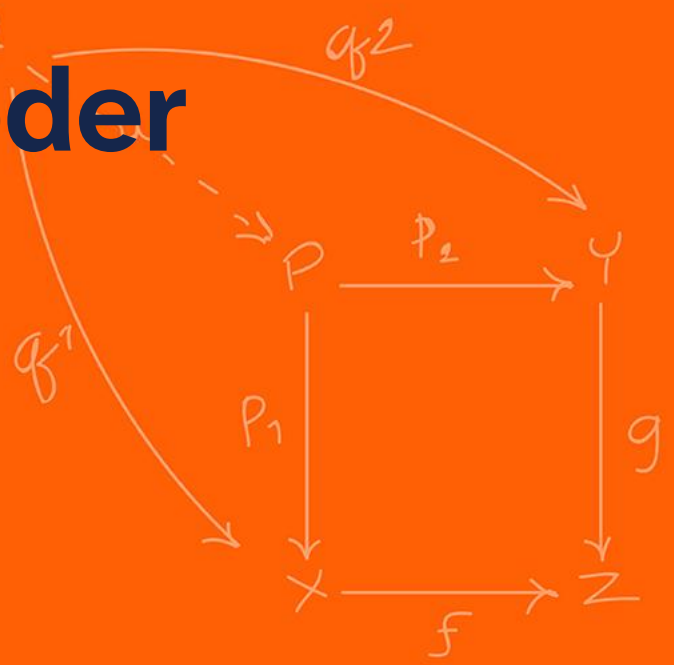








# Absolute Optical Encoder

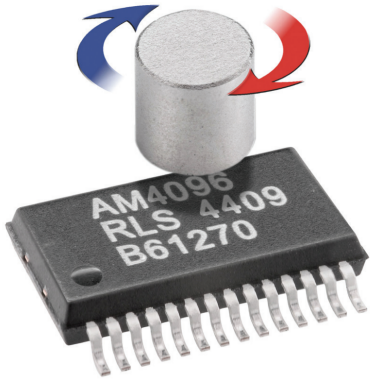


# Absolute Encoder (1/2)

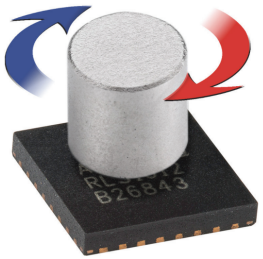
There are multiple protocols for reading data from an optical encoder. We only use **incremental** encoders in this class.

Some protocols use SPI, some use UART or some kind of DAC signal, etc... the only way to be sure is to look at the official documentation!

For the following Magnetic Rotary Encoder (an absolute encoder that uses a magnet on top of the IC), we can extract the data from the SSI communication protocol.




AM4096
(SMD package SSOP28)




AM4096Q
(SMD package QFN32)

https://www.rls.si/eng/am4096-12-bit-rotary-magnetic-encoder-chip

# SSI (1/10)

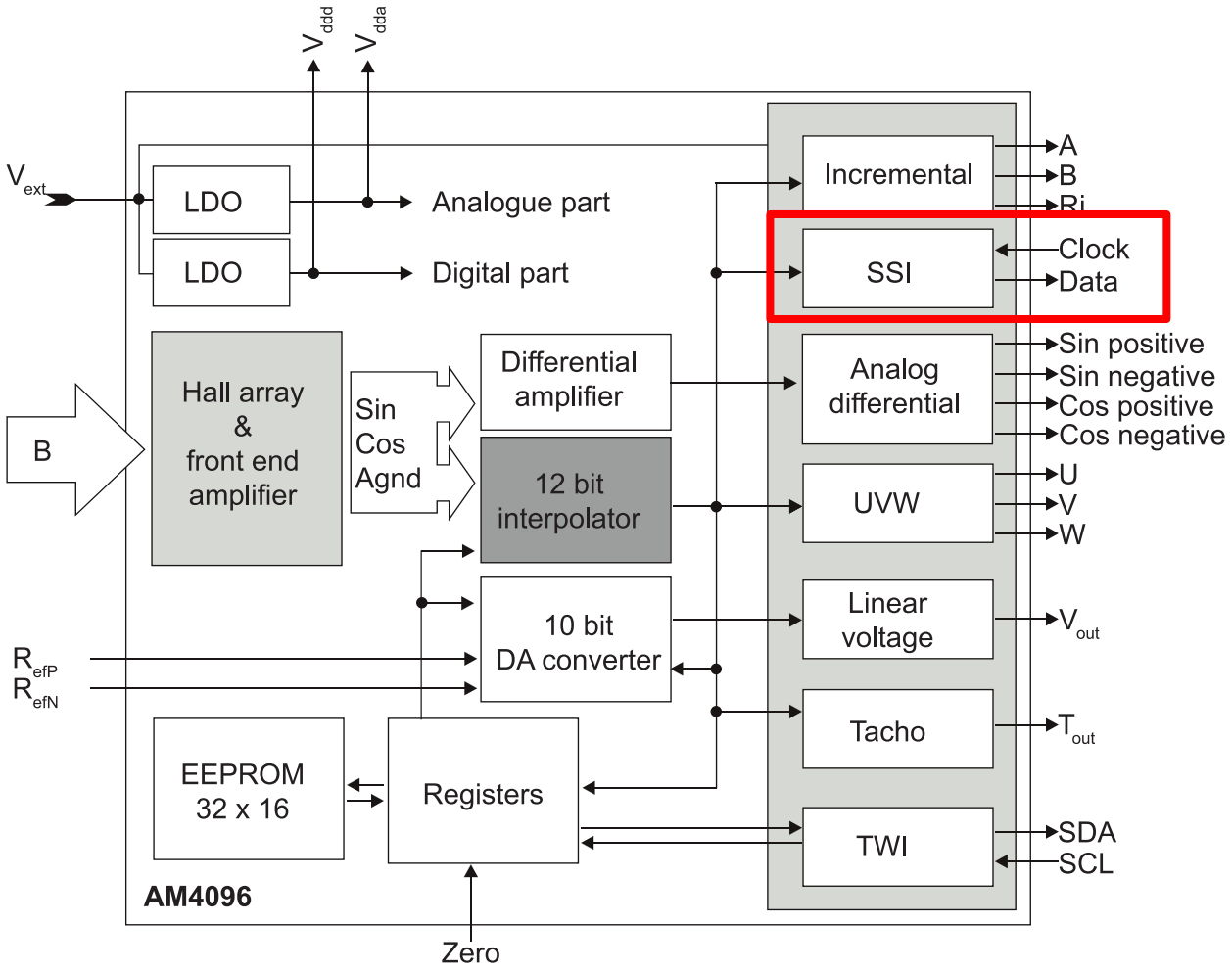



Fig. 1: AM4096 block diagram

# SSI (2/10)

Like UART, start with high, then to start communication you start with a low value. The difference is that you are not sending “data” to ask, you are just sending a clock signal. The sensor is sending its preconfigured “data”.

This is a full-duplex signal (you send and receive at the same time).

The data below is specifically for the [AS5145H – 12 bit angular magnetic encoder IC]

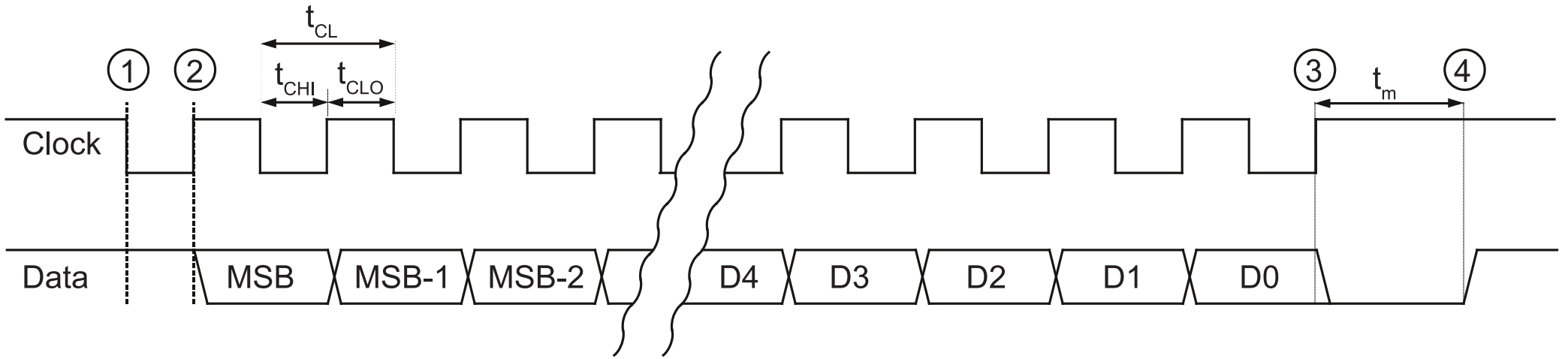
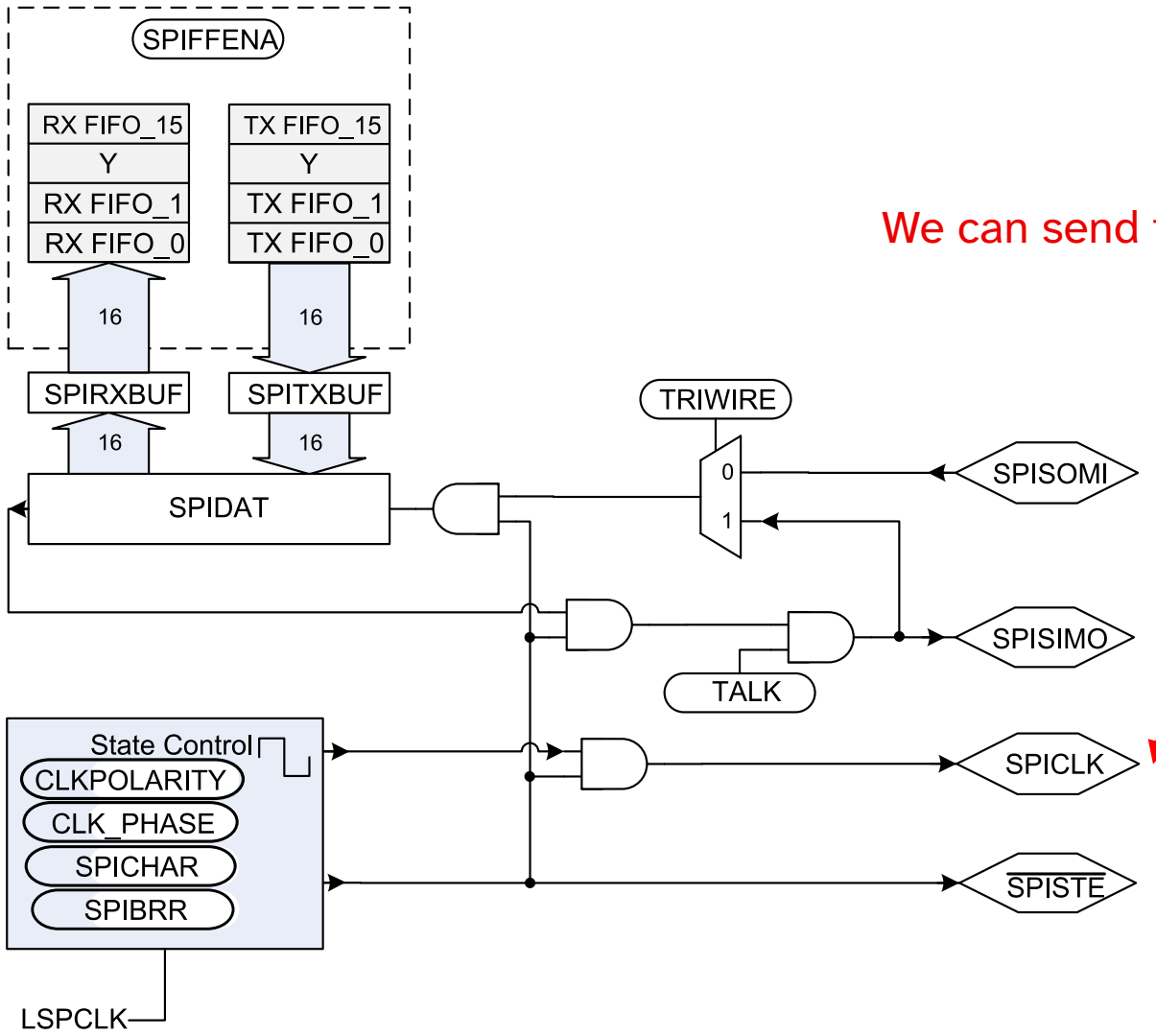



Fig. 12: SSI timing diagram with monoflop timeout

https://look.ams-osram.com/m/5835d15632039c2e

# SSI (3/10)

**Figure 18-5. SPI Module Master Configuration**




We can send the clock signal using SPI

# SSI (4/10)

Communicate through the SPI

```
//need clock polarity = 1 (falling edge)
//collect data on falling edge -> phase = 1 (with delay)
SpiaRegs.SPICCR.bit.SPISWRESET = 0;     //put spi in reset (reset before changin configurations)

SpiaRegs.SPICCR.bit.CLKPOLARITY = 1;    //falling edge clock (starts high)
SpiaRegs.SPICCR.bit.SPICHAR = 7;        //shift length for one character is 8 bits
SpiaRegs.SPICTL.all = 0x000E;           //0000000000001110->clkphase = 1 (with delay), master mode,
//enable talk, spi int disabled, transmit on falling edge

SpiaRegs.SPIBRR.bit.SPI_BIT_RATE = 49;  //set bit rate to 1Mhz so 50Mhz/50?
SpiaRegs.SPIPRI.bit.FREE = 1;           //spi contunes even when emu halted
SpiaRegs.SPISTS.all=0x0000;             //clear all flags
SpiaRegs.SPIFFCT.all=0x00;              //set tx delay to no delays for full duplex ( read and write
at the same time )
```

# SSI (5/10)

```
SpiaRegs.SPIFFTX.bit.TXFIFO = 0;        //put tx fifo in reset and hold there
SpiaRegs.SPIFFTX.bit.TXFFINTCLR = 1;    //clear tx fifo interrupt flag (will not be using)
SpiaRegs.SPIFFRX.bit.RXFIFORESET = 0;   //put rx fifo in reset and hold there
SpiaRegs.SPIFFTX.bit.SPIFFENA = 1;      //spi fifo enhancements enable

SpiaRegs.SPIFFRX.bit.RXFFINTCLR = 1;    //clear receive fifo interrupt flag
SpiaRegs.SPIFFRX.bit.RXFFIL = 3;        //interrupt level is one word
SpiaRegs.SPIFFRX.bit.RXFFIENA = 1;      //rx fifo interrupt enable
SpiaRegs.SPICTL.bit.OVERRUNINTENA = 1;  //overrun interrupt enable
SpiaRegs.SPICTL.bit.SPIINTENA = 1;      //interrupt enable

SpiaRegs.SPIFFTX.bit.TXFIFO = 1;          //release tx fifo to operate
SpiaRegs.SPIFFRX.bit.RXFIFORESET = 1;     //release rx fifo to operate

SpiaRegs.SPIFFTX.bit.SPIRST = 1;        //spi can resume tx or rx (pull out of reset)
SpiaRegs.SPICCR.bit.SPISWRESET = 1;     //pull spi out of reset (enable spi)
//pie and cpu enables for group 6 done in setupSpib (6.1 and 6.2 for spia)
```

# SSI (6/10)

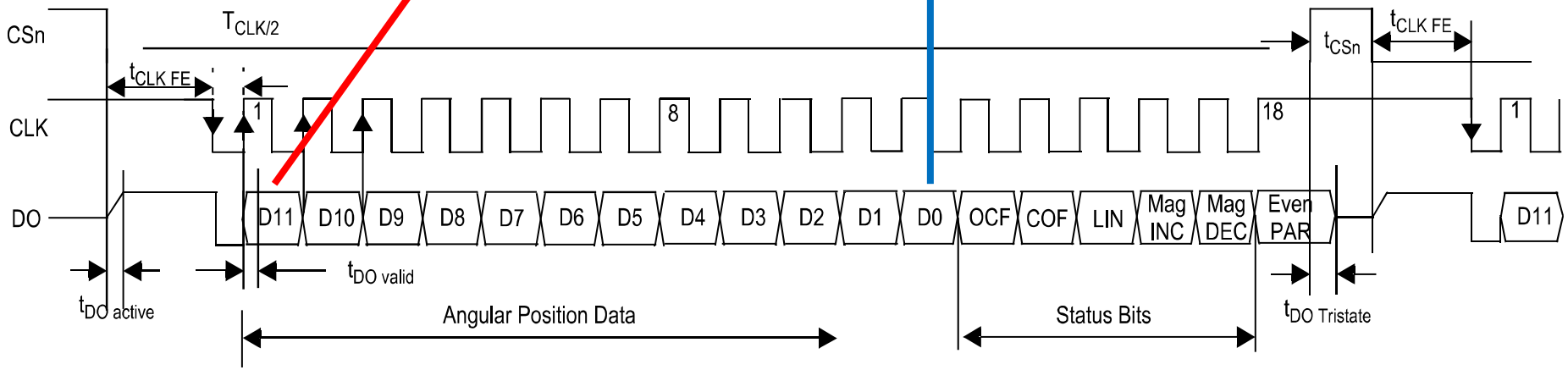
- This sensor send's its information from most significant bit to least significant bit (left-to-right)

0b101000101010

This sensor sends it information as 18 bits of information; however, only on the rising edge of the clock signal!

Since how many bits at minimum to do we need to get the full signal?

18-bits!




https://look.ams-osram.com/m/5835d15632039c2e

# SSI (7/10)

Problem is that SPICHAR (how many bits SPI sends at a time), has a maximum of 16-bits and we need to send 24-bits. So, what do we do?

```
// Send 8-bit words
SpiaRegs.SPICCR.bit.SPICHAR = 7;


// Sending data function
void sendSPIData(void){
    SpiaRegs.SPIFFRX.bit.RXFFIL = 3;                //Going to R/W 3 entries
    GpioDataRegs.GPCCLEAR.bit.GPIO73 = 1;           //pull cs low to get ready to tx/rx

    // since we have SPICHAR=7 (8-bits), we only store the top 8-bits
    // SPI is set to FIFO mode so this works in a special way
    for(i = 0[redacted]; i++) {
        SpiaRegs.SPITXBUF = 0xAA00u; // You need to send something to receive something (does not matter what)!
    }
}
```

# SSI (8/10)

```
void spiaRxFifoIsr(void)
{
    //received data right justified
    spiaRxData[0] = SpiaRegs.SPIRXBUF & 0xFF;
    spiaRxData[1] = SpiaRegs.SPIRXBUF & 0xFF;
    spiaRxData[2] = SpiaRegs.SPIRXBUF & 0xFF;

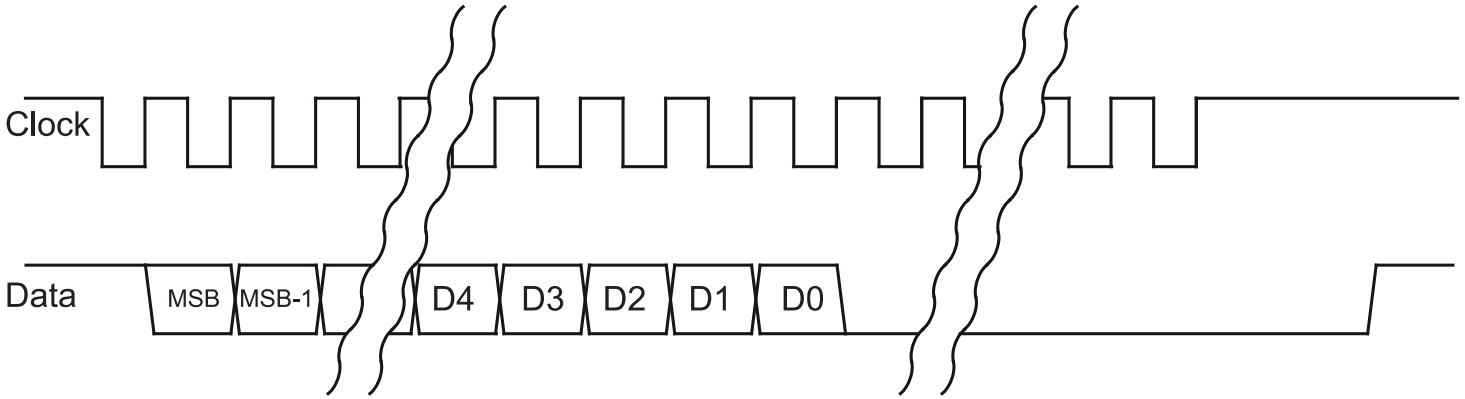
    theta.d = (((unsigned int) (spiaRxData[0] & 0x7F)) << 5) + (((unsigned int) (spiaRxData[1] &
0xF8)) >> 3);
    // ... other math, ignore ...

    SpiaRegs.SPIFFRX.bit.RXFFOVFCLR = 1;  // Clear Overflow flag
    SpiaRegs.SPIFFRX.bit.RXFFINTCLR = 1;  // Clear Interrupt flag
    PieCtrlRegs.PIEACK.all |= 0x20;       // Issue PIE ack
}
```

# Absolute Encoder (2/2)

You need a minimum of 19 low going edges to read in the first always high bit and then the remaining 18 bits.

We are sending 24 bits; the rest does not matter as it gets filtered out.

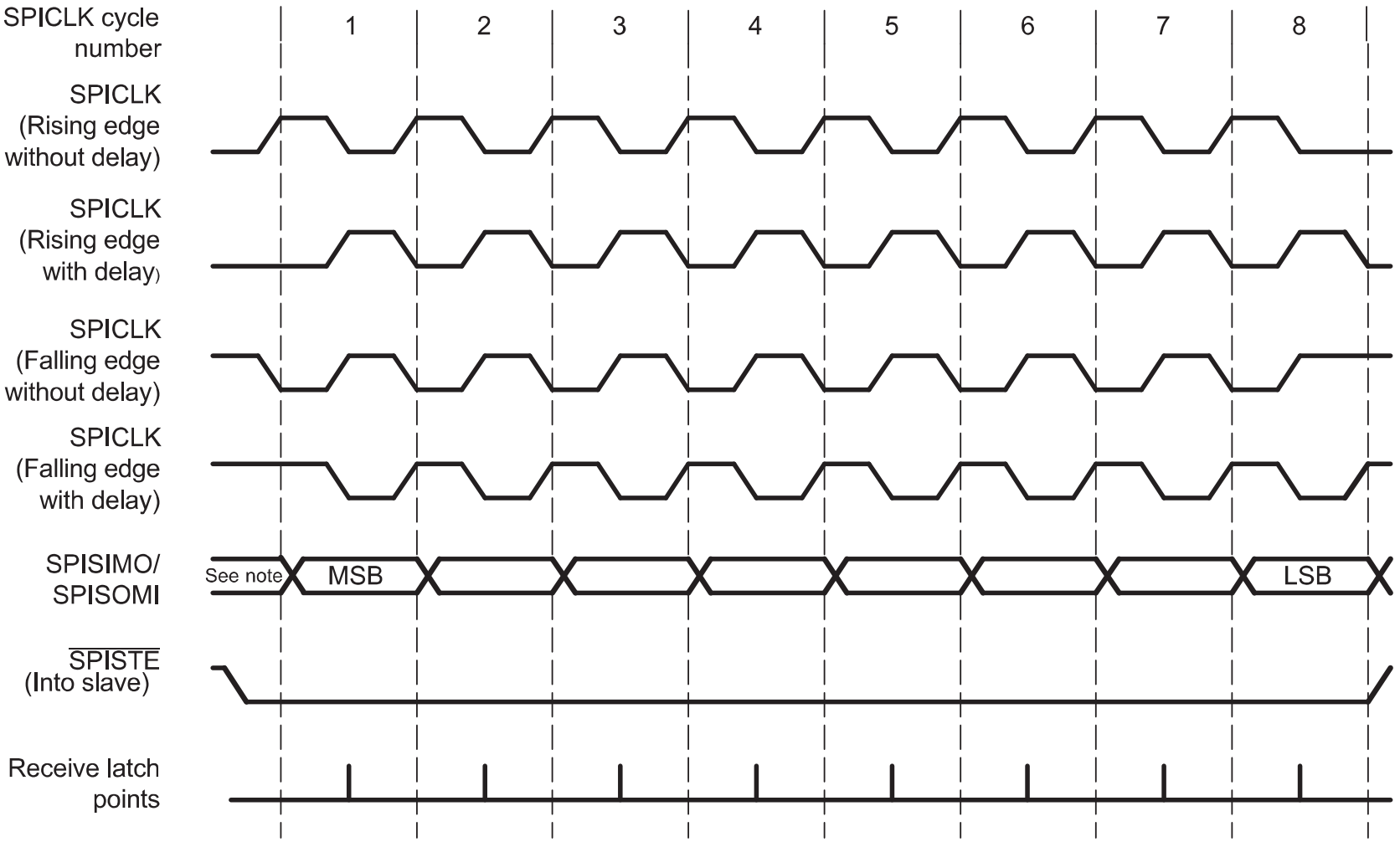



https://www.rls.si/eng/am4096-12-bit-rotary-magnetic-encoder-chip

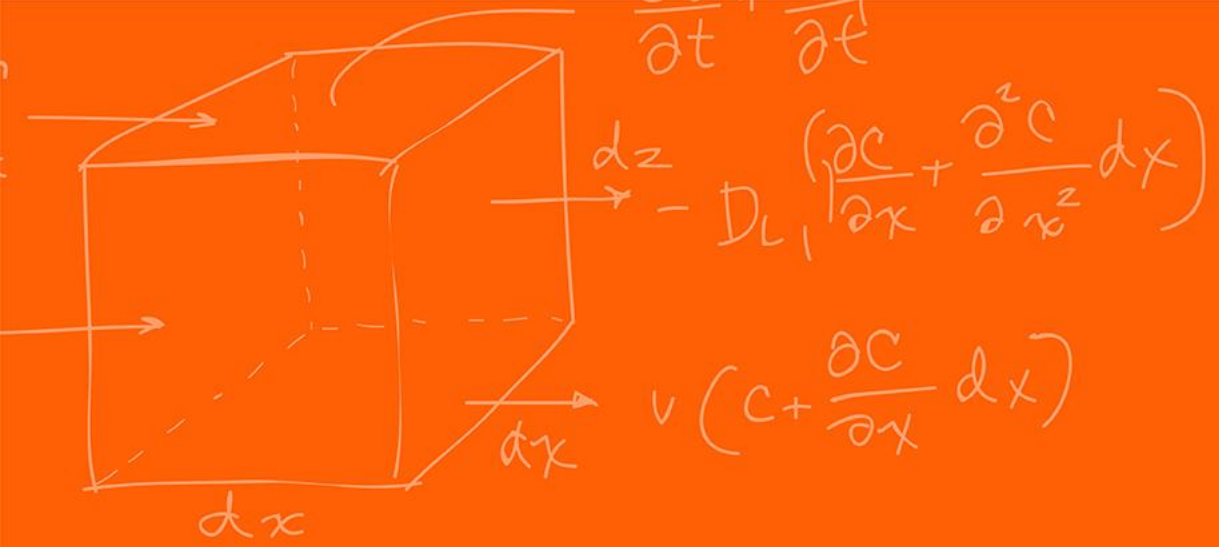
# SSI (9/10)

| Polarity | Phase |
|---|---|
| 0 | 0 |
| 0 | 1 |
| 1 | 0 |
| 1 | 1 |

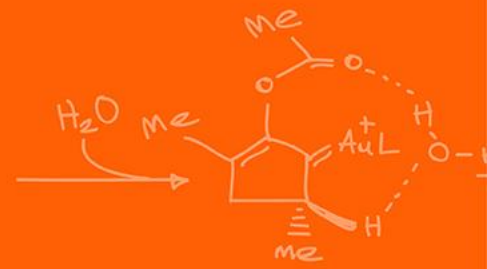
**Figure 18-7. SPICLK Signal Options**




**Note:** Previous data bit

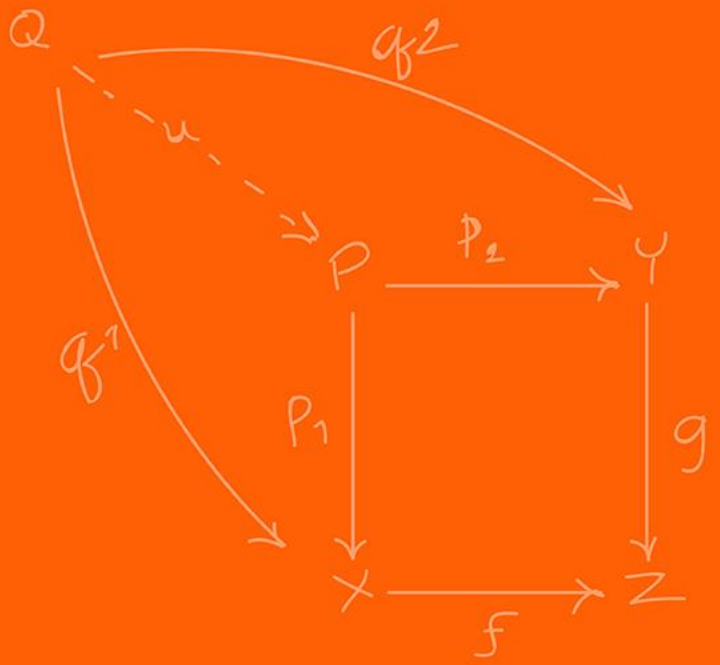








# FIFO (1/4)



# FIFO (2/4)

Previously with the SPI we see that we did 2 weird things:

```
for(i = 0; i < 3; i++) {
   SpiaRegs.SPITXBUF = 0xAA00u;
}
```

And,

```
spiaRxData[0] = SpiaRegs.SPIRXBUF & 0xFF;
spiaRxData[1] = SpiaRegs.SPIRXBUF & 0xFF;
spiaRxData[2] = SpiaRegs.SPIRXBUF & 0xFF;
```

What is weird here?

**We are writing / reading from the same place!** Why would the output be different?!

# FIFO (3/4)

Instead of waiting every interrupt cycle to write long pieces of text to the SPI connection, we can instead use FIFO (First-In-First-Out), which is a buffer queuing system.

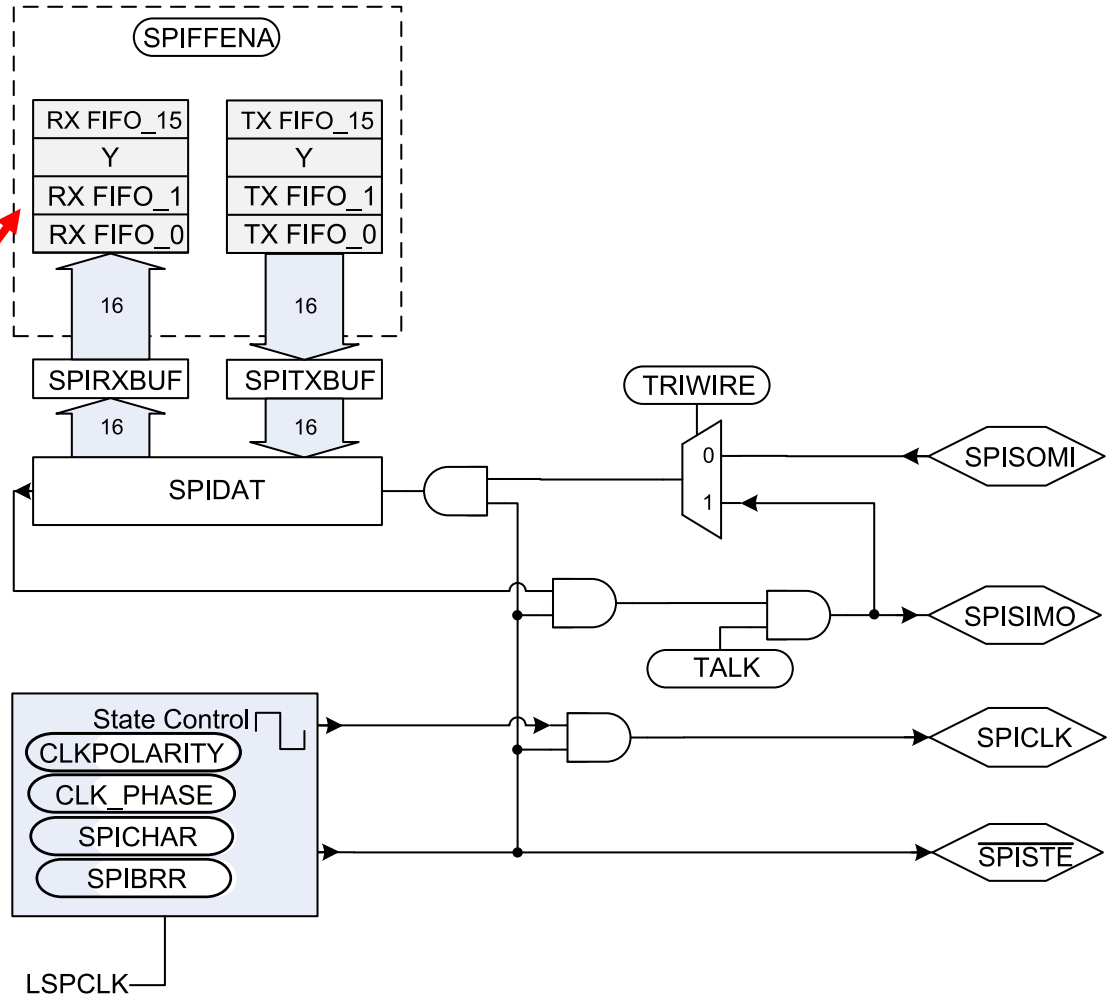
When you fill in one of the buffers RX / TX, the value automatically goes to one of the 16 FIFO slots (instantaneously), to be used later!

This allows you to store, in one function call, 16 FIFO buffer values both for transmit and receive!

Uses only “16 CPU cycles” instead of 16 interrupt calls!

Allows for large read / writes.

**Figure 18-5. SPI Module Master Configuration**



# FIFO (4/4)

Looking back:

```
// Shifts into SPIDATA on the MSB
side
for(i = 0; i < 3; i++) {
   SpiaRegs.SPITXBUF = 0xAA00u;
}
```

```
// Shifts into SPIDATA on the LSB
side
spiaRxData[0] = SpiaRegs.SPIRXBUF &
0xFF;
spiaRxData[1] = SpiaRegs.SPIRXBUF &
0xFF;
spiaRxData[2] = SpiaRegs.SPIRXBUF &
0xFF;
```

For the **TX**, TX FIFO_{0,..,15} start empty.

```
i = 0, TX FIFO_0 = 0xAAu;
i = 1, TX FIFO_{0,1} = 0xAAu;
i = 2, TX FIFO_{0,1,2} = 0xAAu;
```

For the **RX**, RX FIFO_{0,1,2} has data

```
Takes from RX FIFO_0 FIFO_{1,2} has
data
Takes from RX FIFO_1 FIFO_{2} has
data
Takes from RX FIFO_2 FIFO has no more
data
```

(technically the next FIFO is always FIFO_0)

# FIFO – Example 18-1

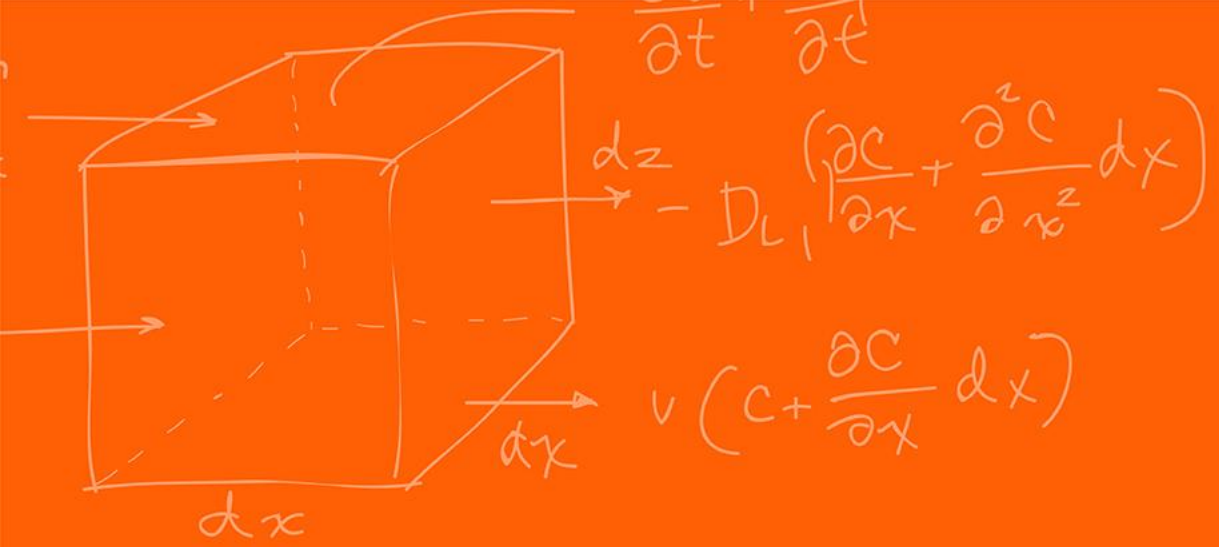
| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **SPIDAT (before transmission)** | | | | | | | | | | | | | | | | |
| | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| **SPIDAT (after transmission)** | | | | | | | | | | | | | | | | |
| (TXed) 0 ← | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | x[(1)] ← (RXed) |
| **SPIRXBUF (after transmission)** | | | | | | | | | | | | | | | | |
| | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | x[(1)] |

(1) x = 1 if SPISOMI data is high; x = 0 if SPISOMI data is low; master mode is assumed.

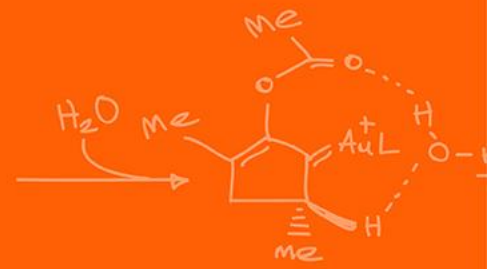
# SSI (10/10)

The four-bit SPICHAR register field specifies the number of bits in the data character (1 to 16). The following statements apply to characters with fewer than 16 bits:

- Data must be left-justified when written to SPIDAT and SPITXBUF.
- Data read back from SPIRXBUF is right-justified.
- SPIRXBUF contains the most recently received character, right-justified, plus any bits that remain from previous transmission(s) that have been shifted to the left (shown in Example 18-1).

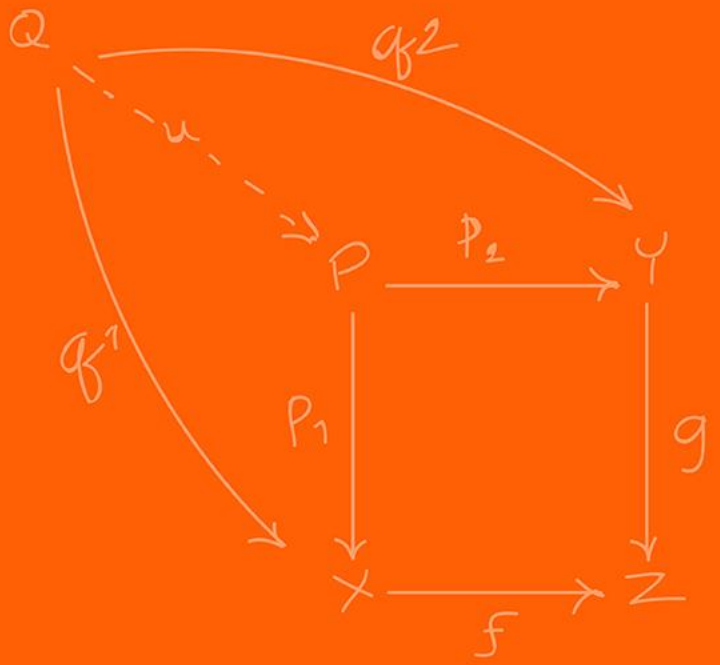








# PWM (1/4)



# How do LEDs become dimmer?

# PWM - Introduction (1/2)

- Stands for Pulse Width Modulation

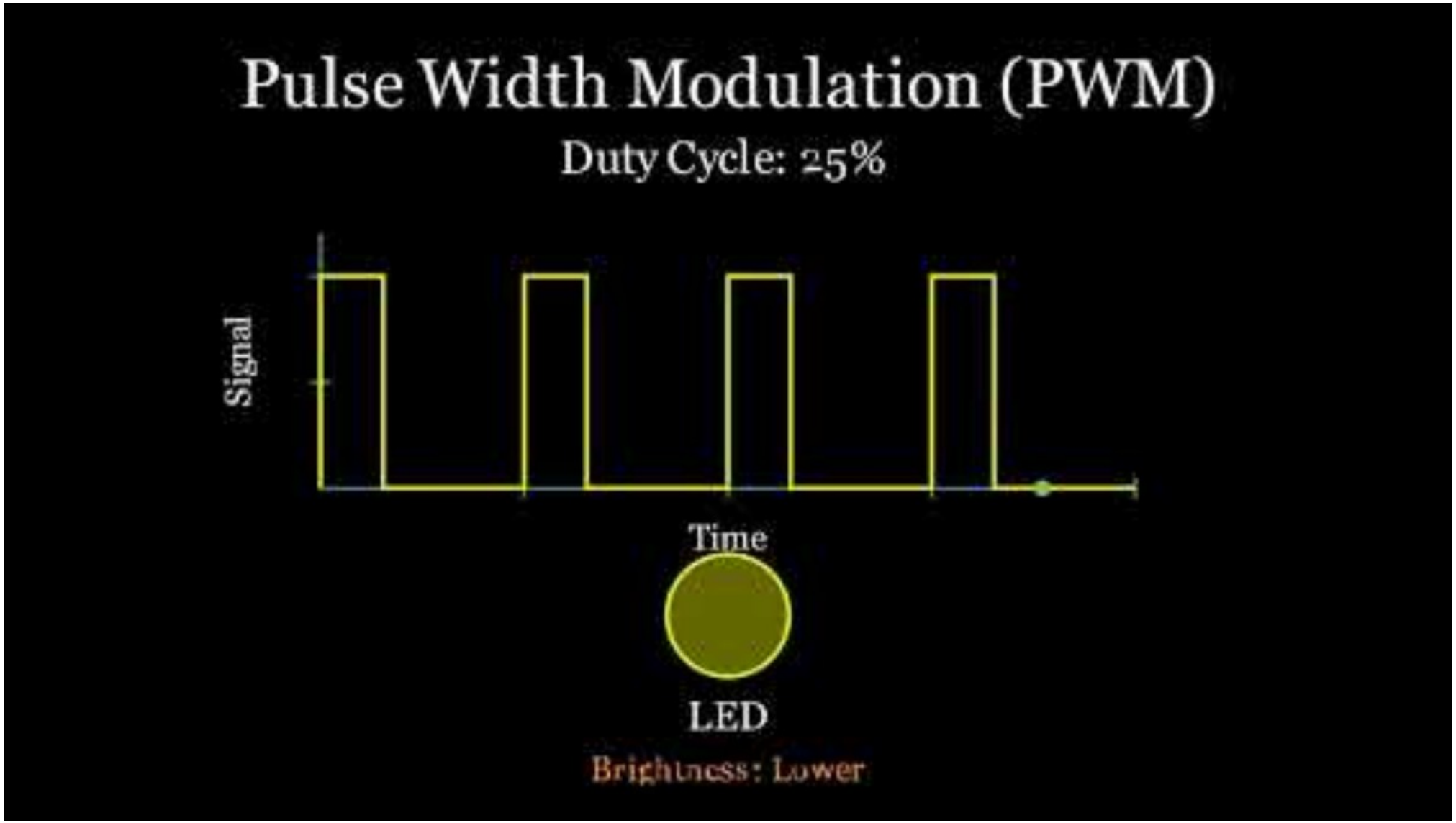
- Rapidly turns the output signal on and off to adjust the *average power delivered*

2 main parameters:

- **Period / carrier frequency** ($T_{pwm}$): The total time of one cycle
- **Duty cycle** ($D$)**:** The fraction of time the signal is high $D = \frac{T_{on}}{T_{pwm}}$

Again, if the period is fast enough, since our eyes only see at 30-40 Hz, we only see the **average** power! Which is why we see a continuous dinner or brighter light!

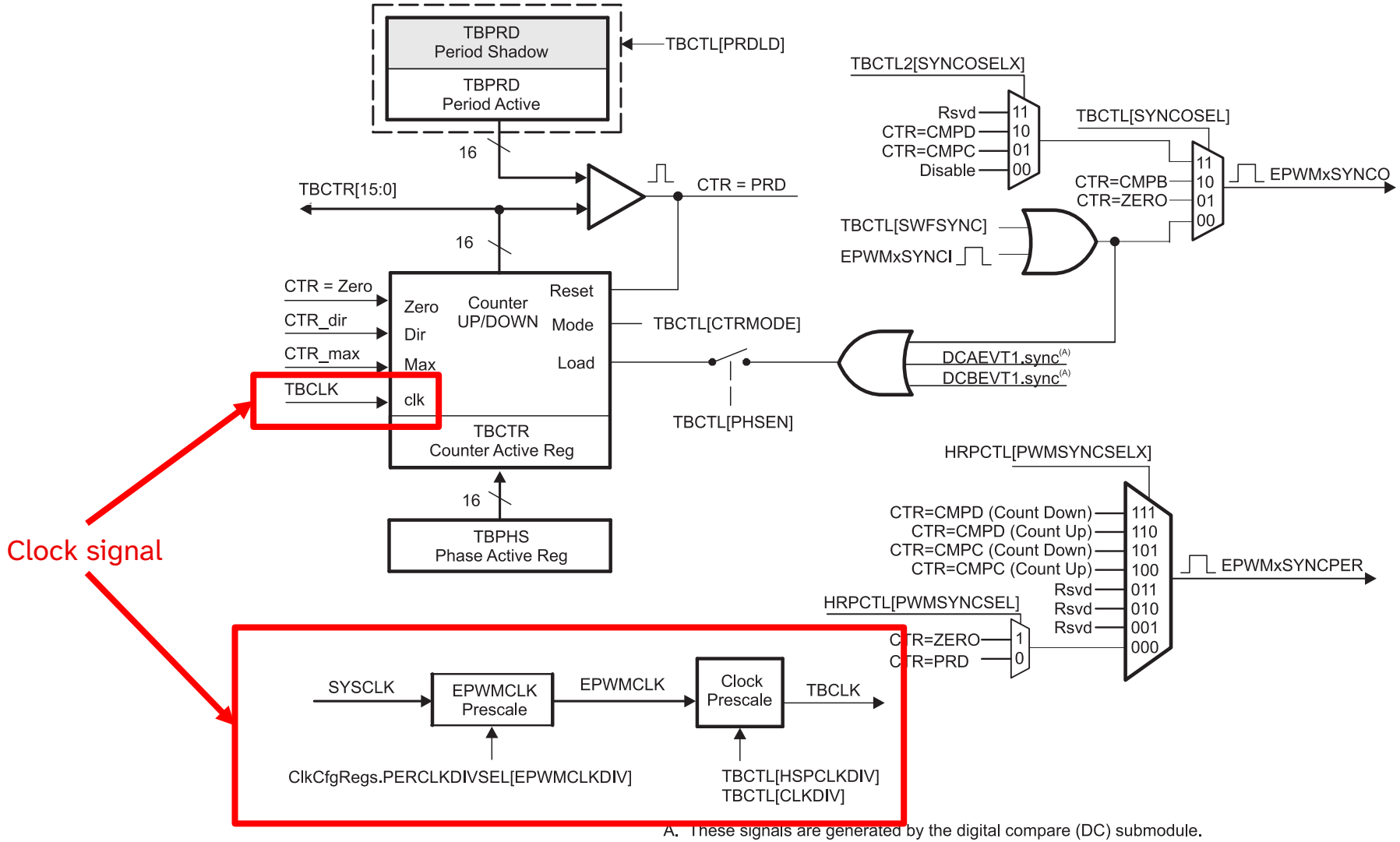
# PWM - Introduction (2/2)




https://youtu.be/IO5RBfq_5Uo?si=KMDtMRWZ-Gs9MQLh

# PWM (2/4)

**Figure 15-5. Time-Base Submodule Signals and Registers**




A. These signals are generated by the digital compare (DC) submodule.

# PWM (3/4)

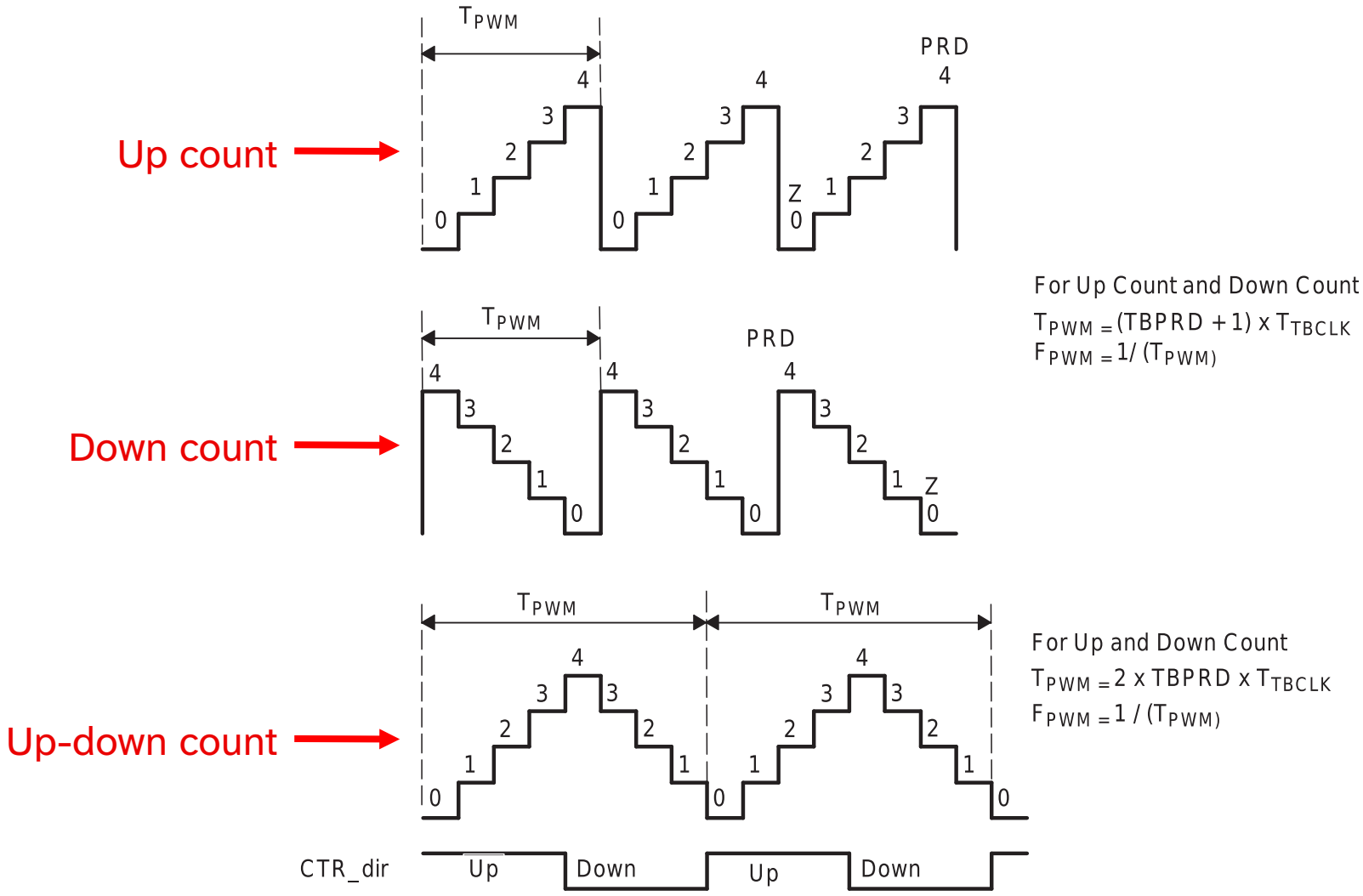
The frequency of PWM events is controlled by the **time-base period** (TBPRD) register and the mode of the time-base counter. The time increment for each step is defined by the **time-base clock** (TBCLK) which is a prescaled version of the ePWM clock (EPWMCLK).

The time-base counter has **three modes** of operation selected by the time-base control register (TBCTL):

- **Up-Down-Count Mode**: In up-down-count mode, the time-base counter starts from zero and increments until the period (TBPRD) value is reached. When the period value is reached, the time-base counter then decrements until it reaches zero.
- **Up-Count Mode**: In this mode, the time-base counter starts from zero and increments until it reaches the value in the period register (TBPRD).
- **Down-Count Mode**: In down-count mode, the time-base counter starts from the period (TBPRD) value and decrements until it reaches zero.

# PWM – Counter mode

**Figure 15-6. Time-Base Frequency and Period**




For Up Count and Down Count

$T_{PWM} = (TBPRD + 1) \times T_{TBCLK}$

$F_{PWM} = 1/(T_{PWM})$

For Up and Down Count

$T_{PWM} = 2 \times TBPRD \times T_{TBCLK}$

$F_{PWM} = 1/(T_{PWM})$

# PWM – Counter-Compare Submodule

The counter-compare submodule takes as input the time-base counter value.

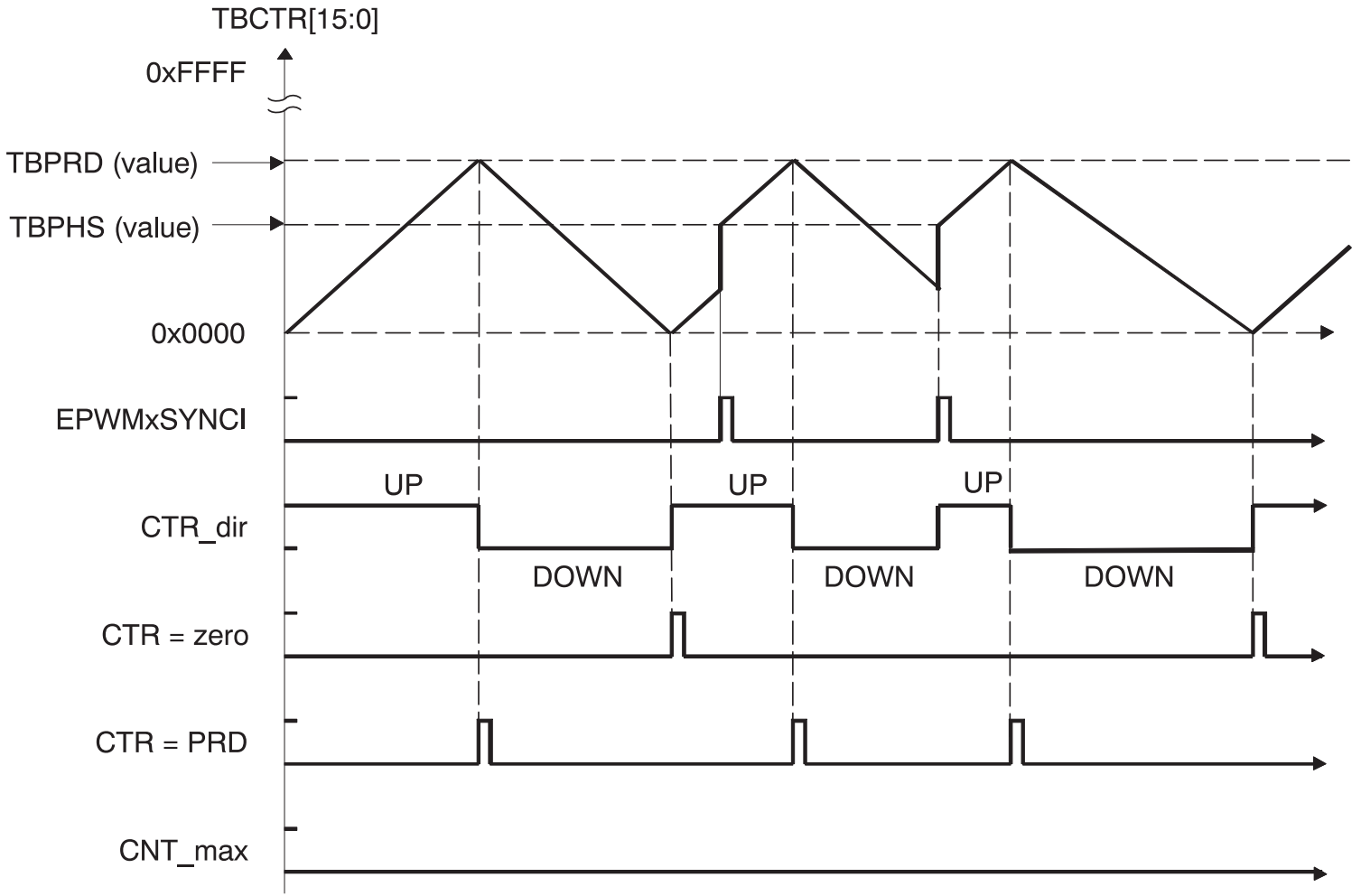
- This value is continuously compared to the counter-compare A (CMPA) counter-compare B (CMPB) counter-compare C (CMPC) and counter-compare D (CMPD) registers.
- When the time-base counter is **equal to one of the compare registers**, the counter-compare unit **generates an appropriate event**.

**Table 15-3. Action-Qualifier Submodule Possible Input Events**

| Signal | Description | Registers Compared |
|---|---|---|
| CTR = PRD | Time-base counter equal to the period value | TBCTR = TBPRD |
| CTR = Zero | Time-base counter equal to zero | TBCTR = 0x00 |
| CTR = CMPA | Time-base counter equal to the counter-compare A | TBCTR = CMPA |
| CTR = CMPB | Time-base counter equal to the counter-compare B | TBCTR = CMPB |
| T1 event | Based on comparator, trip or syncin events | None |
| T2 event | Based on comparator, trip or syncin events | None |
| Software forced event | Asynchronous event initiated by software | |

# PWM (4/4)

**Figure 15-11. Time-Base Up-Down Count Waveforms, TBCTL[PHSDIR = 1] Count Up On Synchronization Event**



# PWM – Duty cycle (1/8)

From the result of the comparison that get triggered, you can then output a signal to the ePWMA or ePWMB signals

The possible actions imposed on outputs EPWMxA and EPWMxB are:

- **Set High**: Set output EPWMxA or EPWMxB to a high level.
- **Clear Low**: Set output EPWMxA or EPWMxB to a low level.
- **Toggle**: Toggle EPWMxA or EPWMxB
- **Do Nothing**.

Allows you create complex signals.

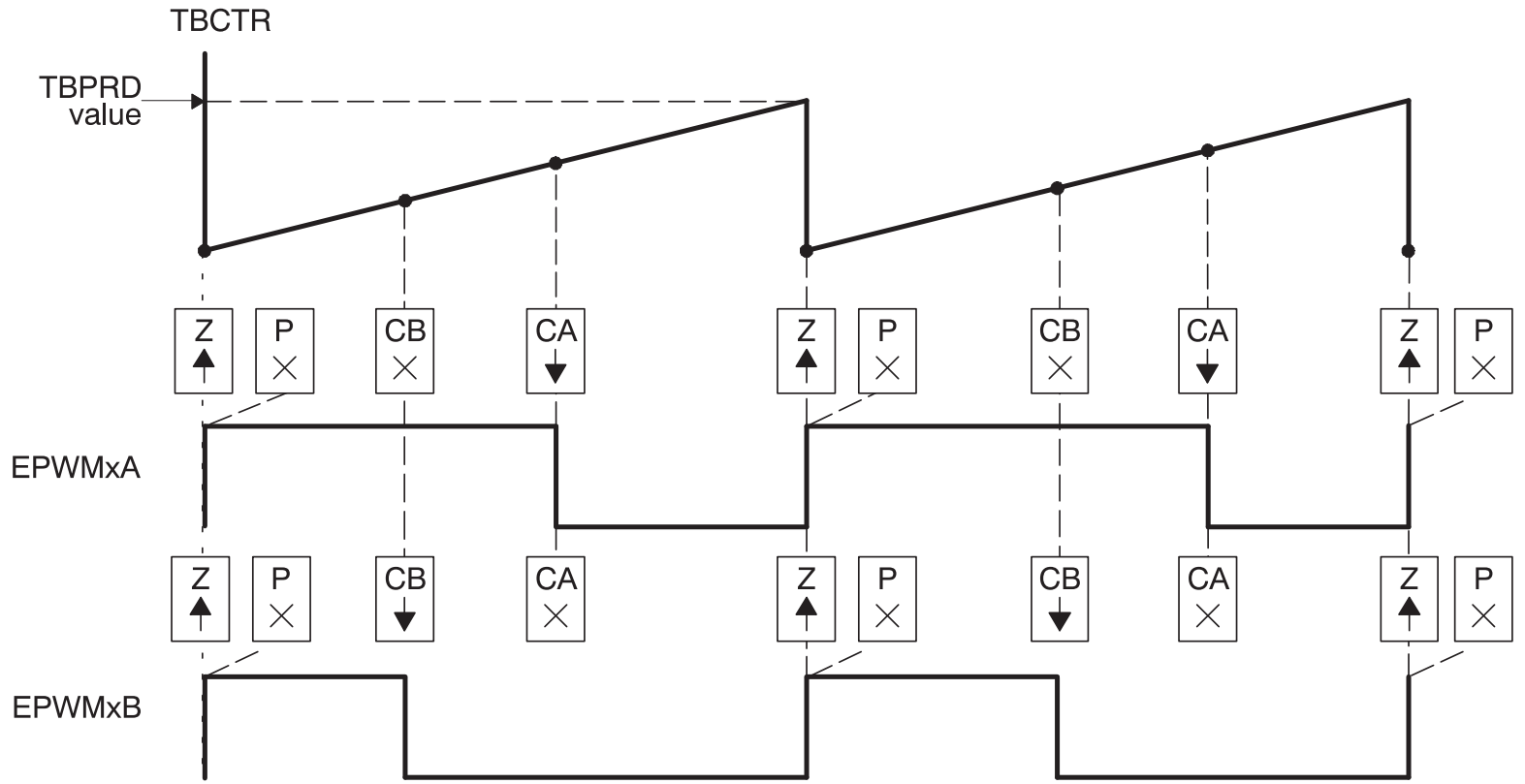
# PWM – Duty cycle (2/8)

Each registry bits allows you to configure the different comparison events

```
struct AQCTLA_BITS {                    // bits description
    Uint16 ZRO:2;                       // 1:0 Action Counter = Zero
    Uint16 PRD:2;                       // 3:2 Action Counter = Period
    Uint16 CAU:2;                       // 5:4 Action Counter = Compare A
Up
    Uint16 CAD:2;                       // 7:6 Action Counter = Compare A
Down
    Uint16 CBU:2;                       // 9:8 Action Counter = Compare B
Up
    Uint16 CBD:2;                       // 11:10 Action Counter = Compare
B Down
    Uint16 rsvd1:4;                     // 15:12 Reserved
};
```

# PWM – Duty cycle (3/8)

**Figure 15-25. Up, Single Edge Asymmetric Waveform, With Independent Modulation on EPWMxA and EPWMxB—Active High**




A PWM period = (TBPRD + 1 ) × $T_{TBCLK}$

B Duty modulation for EPWMxA is set by CMPA, and is active high (that is, high time duty proportional to CMPA).

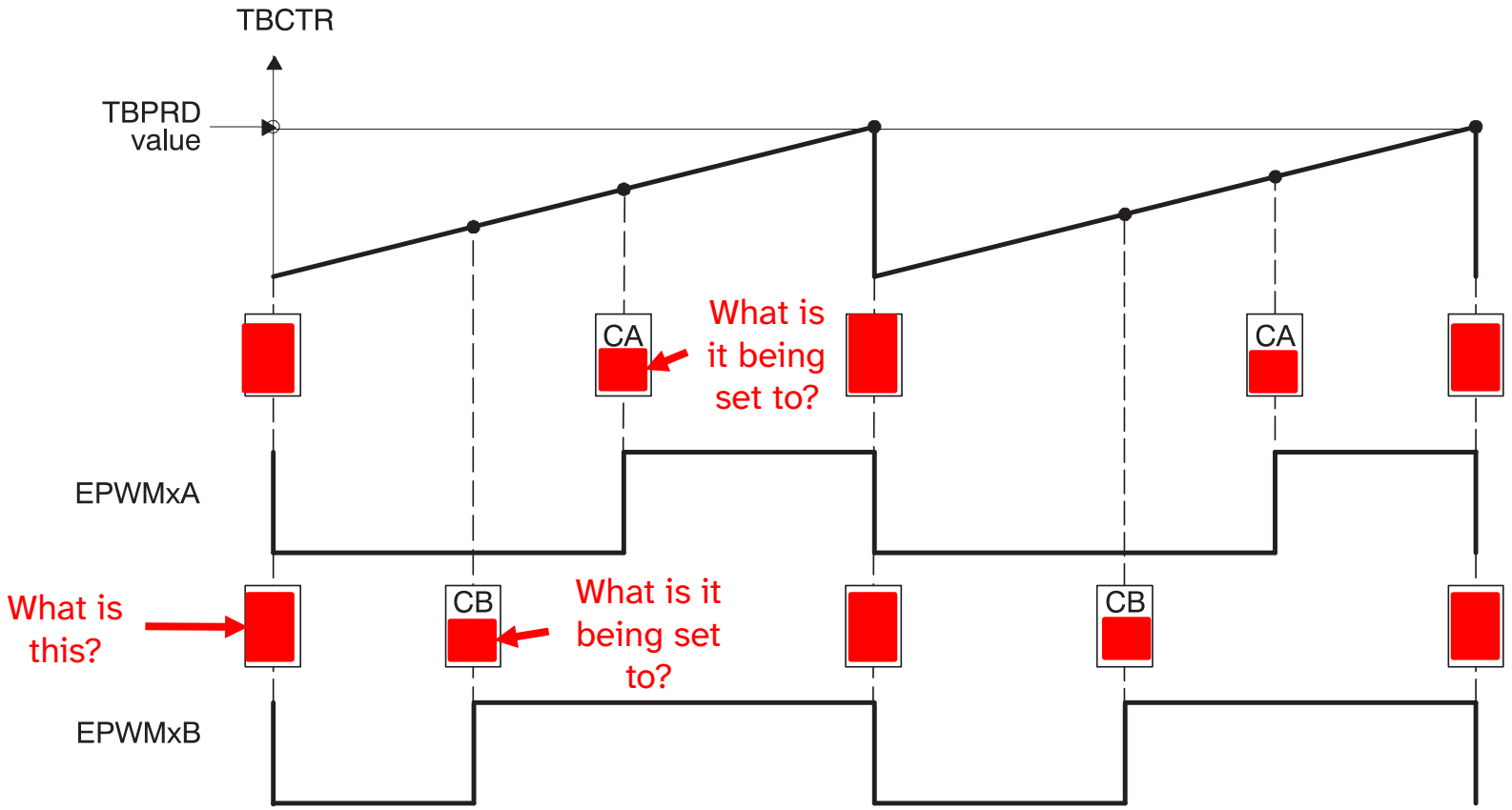
C Duty modulation for EPWMxB is set by CMPB and is active high (that is, high time duty proportional to CMPB).

D The "Do Nothing" actions ( X ) are shown for completeness, but will not be shown on subsequent diagrams.

E Actions at zero and period, although appearing to occur concurrently, are actually separated by one TBCLK period. TBCTR wraps from period to 0000.

# PWM – Duty cycle (4/8)

**Figure 15-26. Up, Single Edge Asymmetric Waveform With Independent Modulation on EPWMxA and EPWMxB—Active Low**




A PWM period = (TBPRD + 1 ) × $T_{TBCLK}$

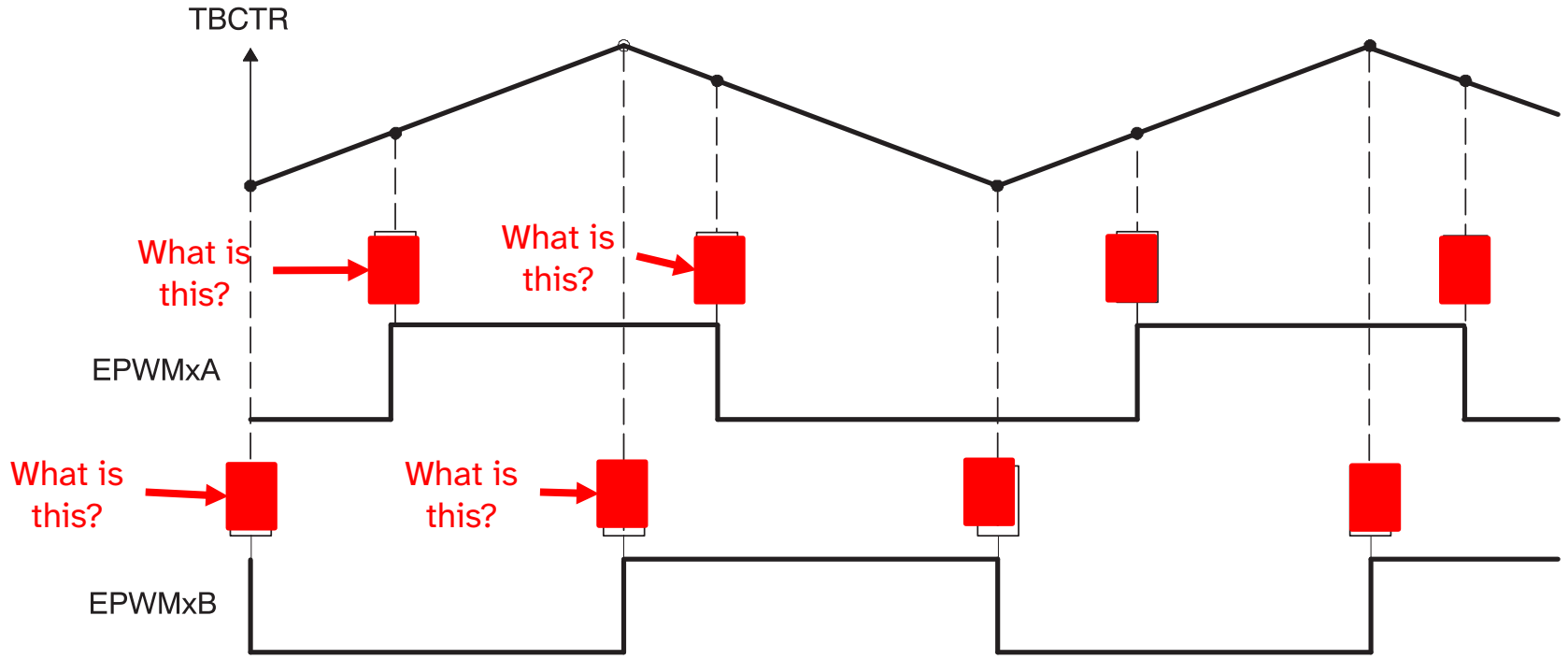
B Duty modulation for EPWMxA is set by CMPA, and is active low (that is, the low time duty is proportional to CMPA).

C Duty modulation for EPWMxB is set by CMPB and is active low (that is, the low time duty is proportional to CMPB).

D Actions at zero and period, although appearing to occur concurrently, are actually separated by one TBCLK period. TBCTR wraps from period to 0000.

# PWM – Duty cycle (5/8)

**Figure 15-30. Up-Down-Count, Dual Edge Asymmetric Waveform, With Independent Modulation on EPWMxA—Active Low**




A PWM period = 2 × TBPRD × TBCLK

B Rising edge and falling edge can be asymmetrically positioned within a PWM cycle. This allows for pulse placement techniques.

C Duty modulation for EPWMxA is set by CMPA and CMPB.

D Low time duty for EPWMxA is proportional to (CMPA + CMPB).

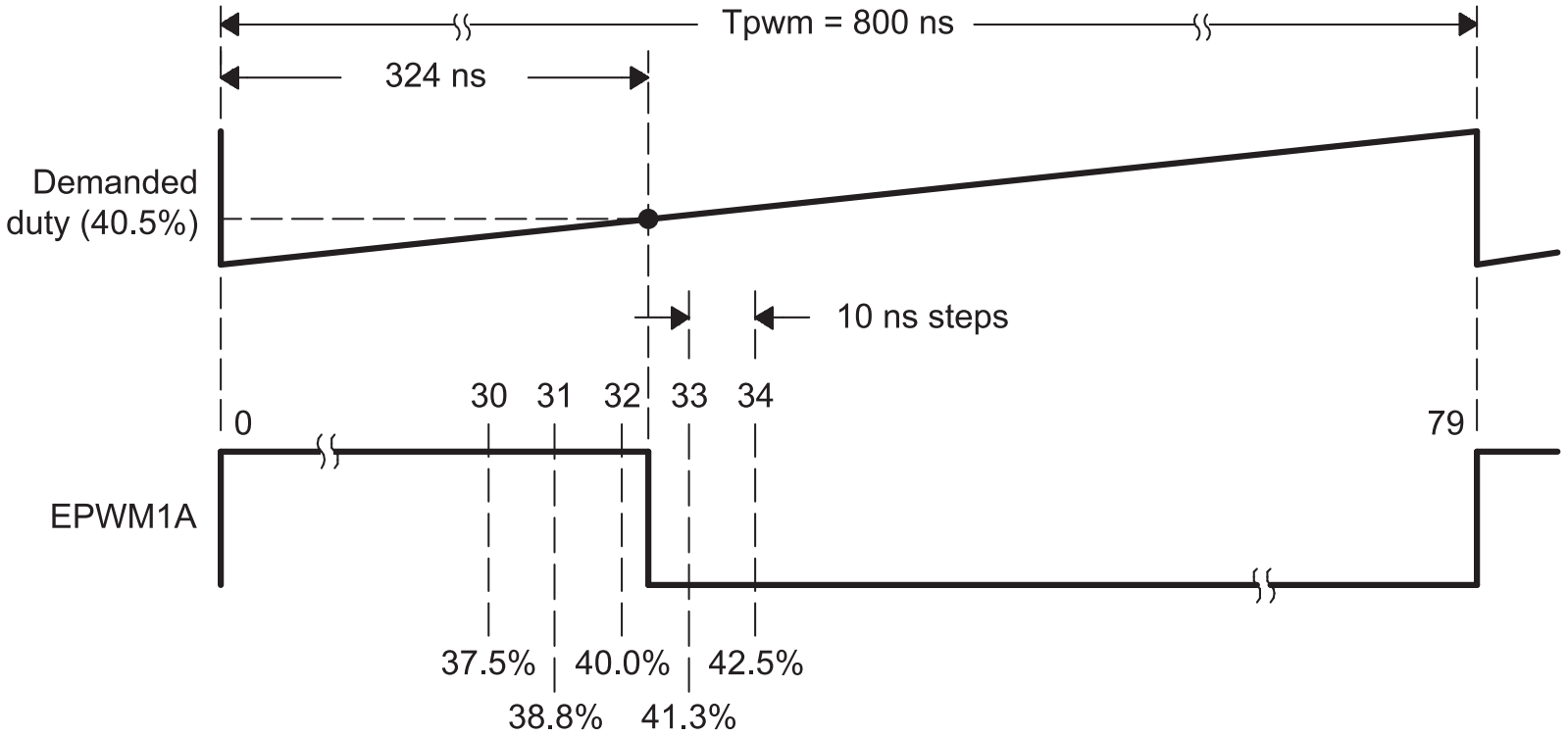
E To change this example to active high, CMPA and CMPB actions need to be inverted (that is, Clear on CMPA, Set on CMPB).

F Duty modulation for EPWMxB is fixed at 50% (utilizes spare action resources for EPWMxB).

# PWM – Duty cycle (6/8)

For the PWM you can set the clock divide to be very small such that your $T_{pwm}$ is very small; however, the problem is that that reduces your duty cycle resolution. As you will run into system clock resolution limitation.

**Figure 15-84. Required PWM Waveform for a Requested Duty = 40.5%**



# PWM – Duty cycle (7/8)

```
EALLOW;
//    TBCTL
EPwm12Regs.TBCTL.bit.FREE_SOFT = 0x2;  // Free Soft emulation = Free Run
EPwm12Regs.TBCTL.bit.CLKDIV = 0;       // Clock divide = 1
EPwm12Regs.TBCTL.bit.CTRMODE = 1;      // Count down Mode
EPwm12Regs.TBCTL.bit.PHSEN = 0;        // Do not load time time-base counter from the time-base phase
registry

//    TBCTR
EPwm12Regs.TBCTR = 0;                  // Start Timer at zero

//    TBPRD
//    Set period of the PWM to 2kHz (500 us)
//    Counter frequency is 50MHz
EPwm12Regs.TBPRD = 25000;

//    CMPA
EPwm12Regs.CMPA.bit.CMPA = 12500;       // Duty cycle at 50%, TBPRD / 2

EPwm12Regs.AQCTLA.bit.CAD = 0x1;       // PWM12A output pin is set low when CMPA is reached
EPwm12Regs.AQCTLA.bit.PRD = 0x2;       // the pin be set high when the TBCTR register is Period

EPwm12Regs.TBPHS.bit.TBPHS = 0;
EDIS;

GpioCtrlRegs.GPAPUD.bit.GPIO22 = 1; // For EPW12A
```

# PWM – Duty cycle (8/8)

```
EPwm12Regs.TBPRD = 100;
EPwm12Regs.CMPA.bit.CMPA = 50;
```

What is the duty cycle?
50%

```
EPwm12Regs.TBPRD = 100;
EPwm12Regs.CMPA.bit.CMPA = 25;
```

What is the duty cycle?
25%

```
EPwm12Regs.TBPRD = 100;
EPwm12Regs.CMPA.bit.CMPA = 75;
```

What is the duty cycle?
75%

# PWM – DAC

It is also possible to use PWM as a DAC! By selecting the proper duty cycle, any DAC output voltage can be obtained. You need to remove the high-frequency signals to make this work.

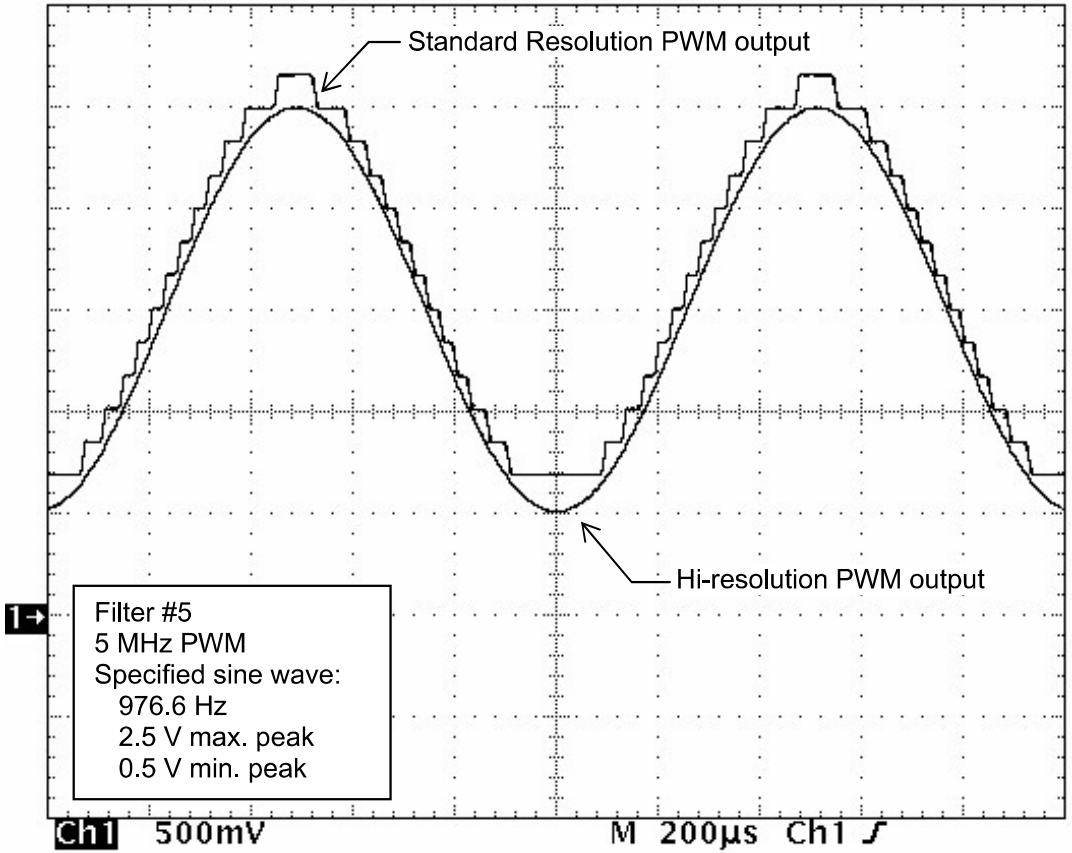
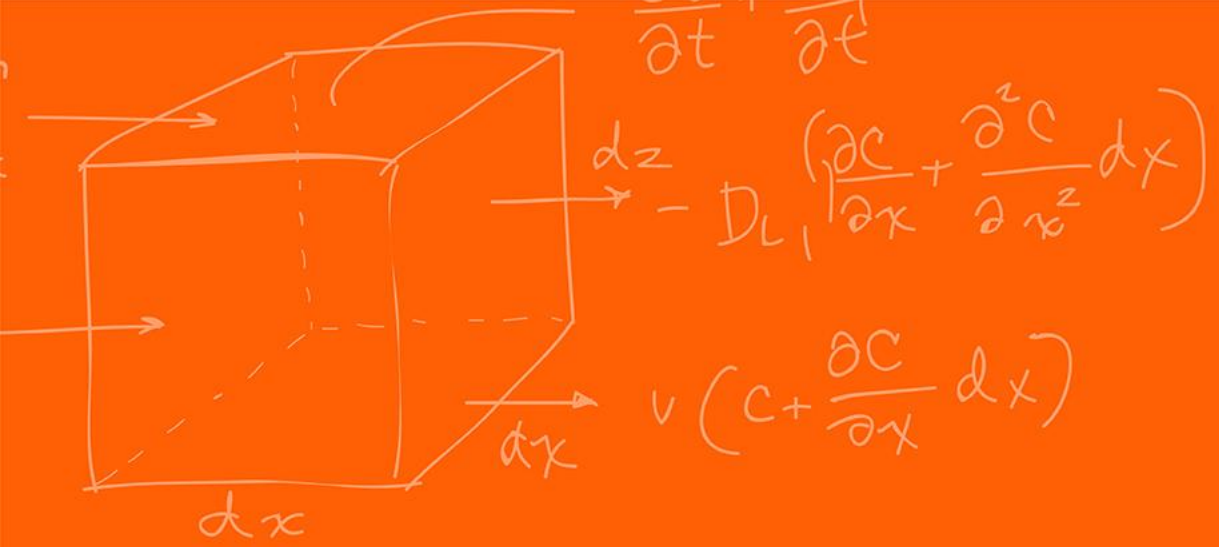
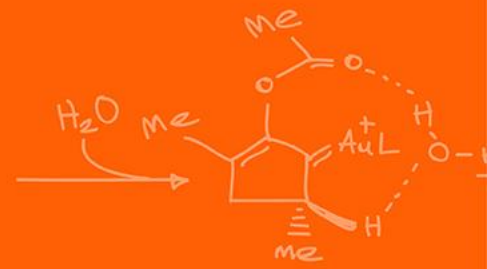



**Figure 17. Filter #5 Output of PWM Sine Wave**

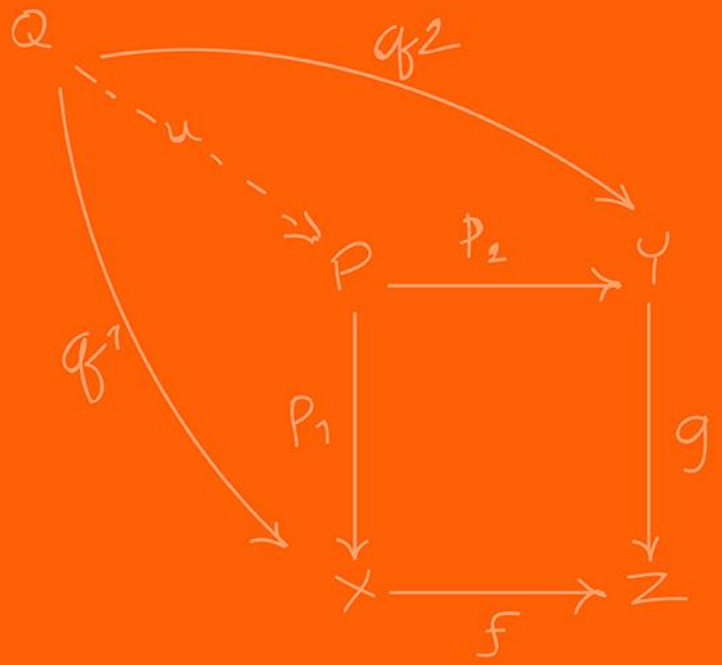




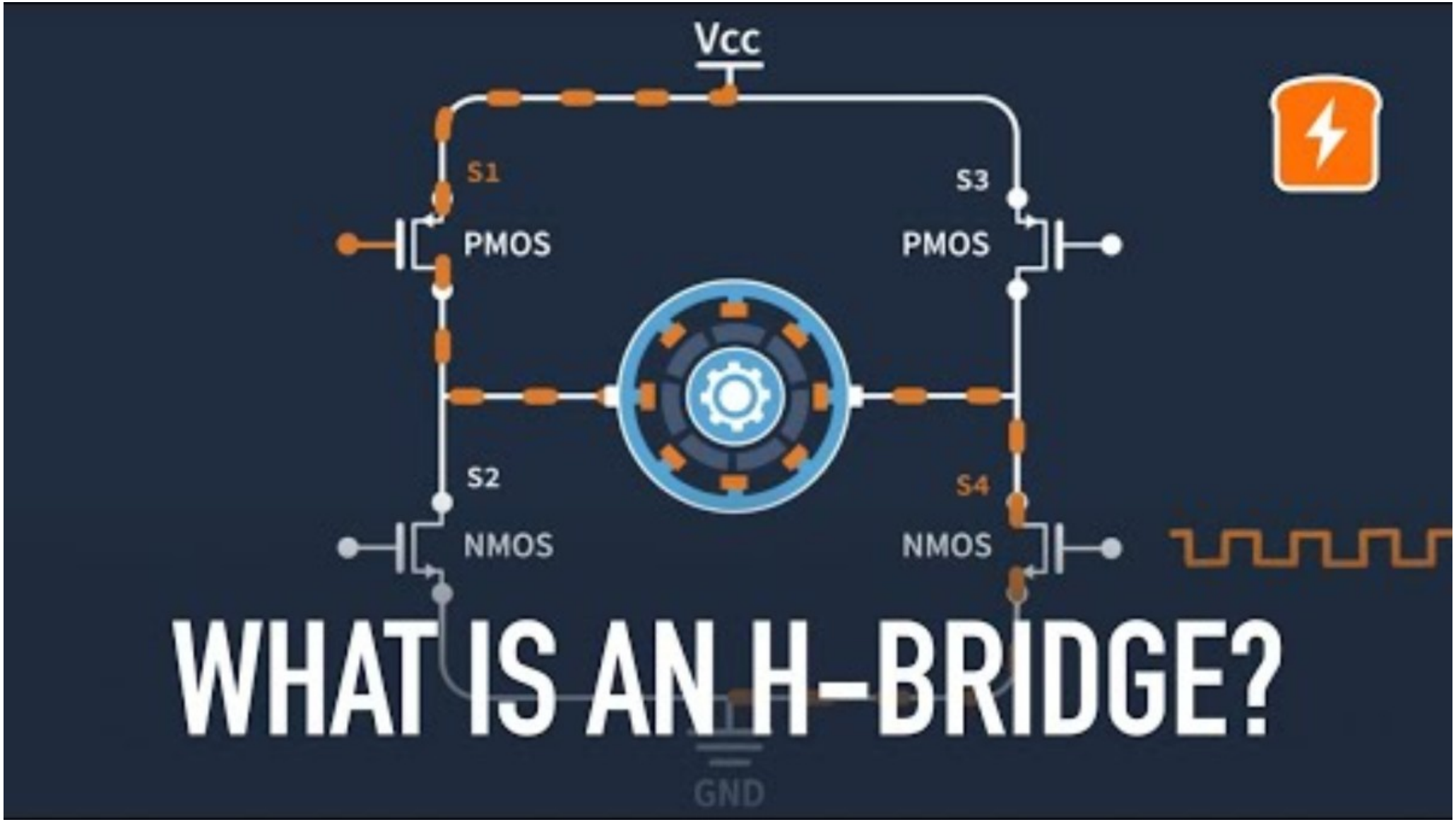




# Motor / Servo

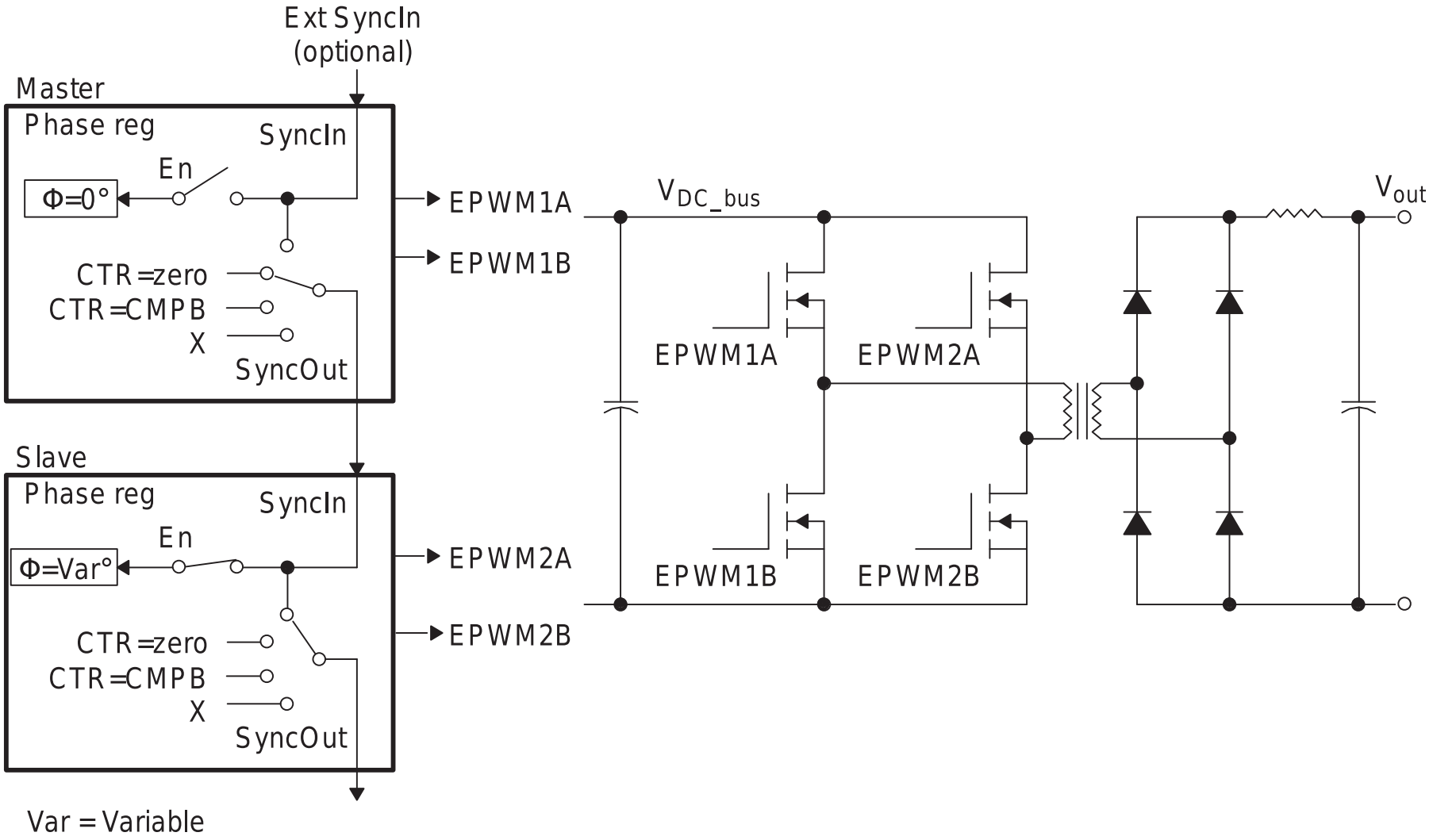


# H-Bridge (1/2)



# H-Bridge (2/2)

**Figure 15-72. Controlling a Full-H Bridge Stage ($F_{PWM2} = F_{PWM1)}$**



# H bridge (1/2)

For the H-bridge chip that we will be using in class, the work has been simplified for you where, you can control the velocity and direction from a single PWM signal.

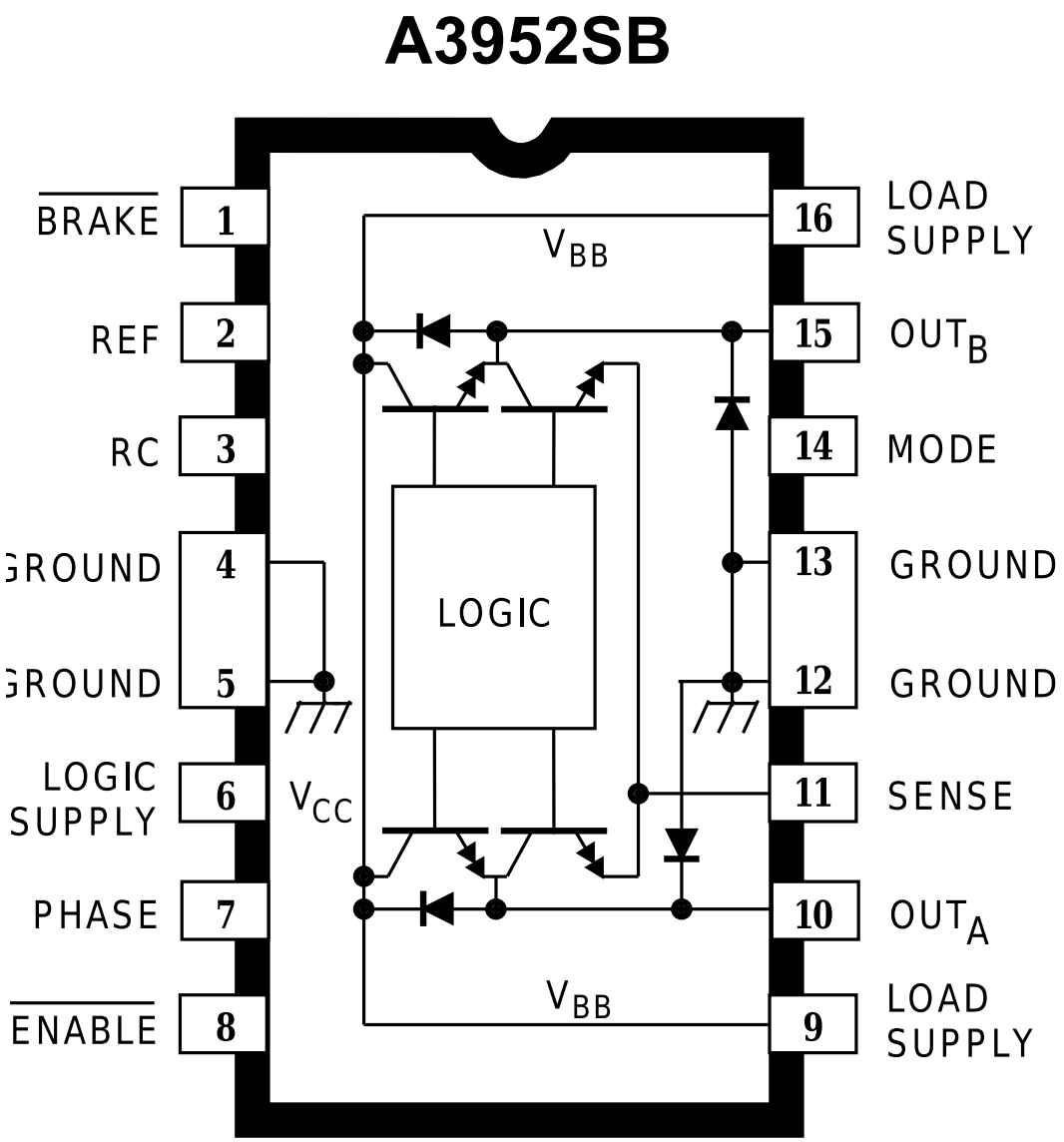
The **duty cycle** from the PWM is what determines the motor's behavior!

- 0% = turns the motor to go **backwards**
- 50% = turns the motor forwards and backwards (makes it so that the motor is stationary)
- 100% = turns the motor to go **forward**

What happens if we set a duty cycle of 75%?

We are going 50% of the maximum forward speed!

# H bridge (2/2)

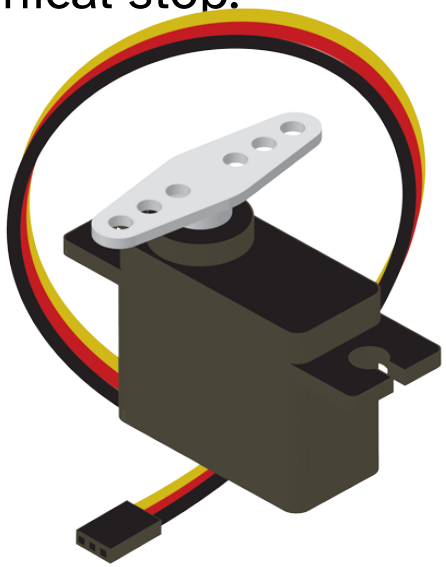

A3952SB
BRAKE
1
REF
2
RC
3
GROUND
4
GROUND
5
LOGIC SUPPLY
6
PHASE
7
ENABLE
8
LOGIC
VBB
VCC
VBB
16
LOAD SUPPLY
15
OUTB
14
MODE
13
GROUND
12
GROUND
11
SENSE
10
OUTA
9
LOAD SUPPLY

# RC Servo (1/2)

RC Servos are position-controlled devices (not continuous rotation) controlled by a specific PWM protocol.

2-types of servo motor,

- **Closed loop**: physically limited to 0-180 degrees (usually) rotation (with an internal pin), you want to be careful to not go to the extremes too often, it is a mechanical stop.
- **Open loop**: can do a full 360-degree rotation.



https://www.sparkfun.com/servos

3 wires:

- **Power**
- **GND**
- **Control**: where the PWM signal is injected

# RC Servo (2/2)

Maps PWM to desired orientation. For specific servo check the datasheet for:

- For the specific period
  - Typically, 50Hz if not listed, which means a period of 20 ms = 20,000 µs
- Operating range and duty cycle mapping.

# RC Servo – Example Datasheet

Using a PWM period of 20 ms, what is 90 degrees (neutral), 50 and 130?

## ANNOUNCED SPECIFICATIONS OF HS–85MG+ MIGHT MICRO METAL GEAR SERVO

| | | |
|---|---|---|
| 1.TECHNICAL VALUES | | |
| CONTROL SYSTEM | :+PULSE WIDTH CONTROL 1500usec NEUTRAL | |
| OPERATING VOLTAGE RANGE | :4.8V TO 6.0V | |
| OPERATING TEMPERATURE RANGE | :-20° C TO +60° C | |
| TEST VOLTAGE | :AT 4.8V AT 6.0V | |
| OPERATING SPEED | :0.16sec/60° AT NO LOAD | 0.14sec/60° AT NO LOAD |
| STALL TORQUE | :3.0kg.cm(41.66oz.in) | 3.5kg.cm(48.60oz.in) |
| OPERATING ANGLE | :40%ONE SIDE PULSE TRAVELING 400usec | |
| DIRECTION | :CLOCK WISE/PULSE TRAVELING 1500 TO 1900usec | |
| CURRECT DRAIN | :8mA/IDLE AND 240mA NO LOAD RUNNING | |
| DEAD BAND WIDTH | :8usec | |
| CONNECTOR WIRE LENGTH | :160mm (6.29in) | |
| DIMENSIONS | :29x13x30mm (1.14x0.51x1.18in) | |
| WEIGHT | :21.9g (0.77oz) | |

Represents 90 degrees

50 = 1100 us = 5.5%
130 = 1900 us = 9.5%

Tells you which direction is clockwise ( 130 degrees )

This servo is only able to do $90 \pm 40$ and not the full 0-180 degrees!

https://www.sparkfun.com/servo-hitec-hs-85mg-micro-size.html

# Piezo Buzzer (1/2)

Instead of varying the duty cycle, we define notes by the frequency that we put into the buzzer!

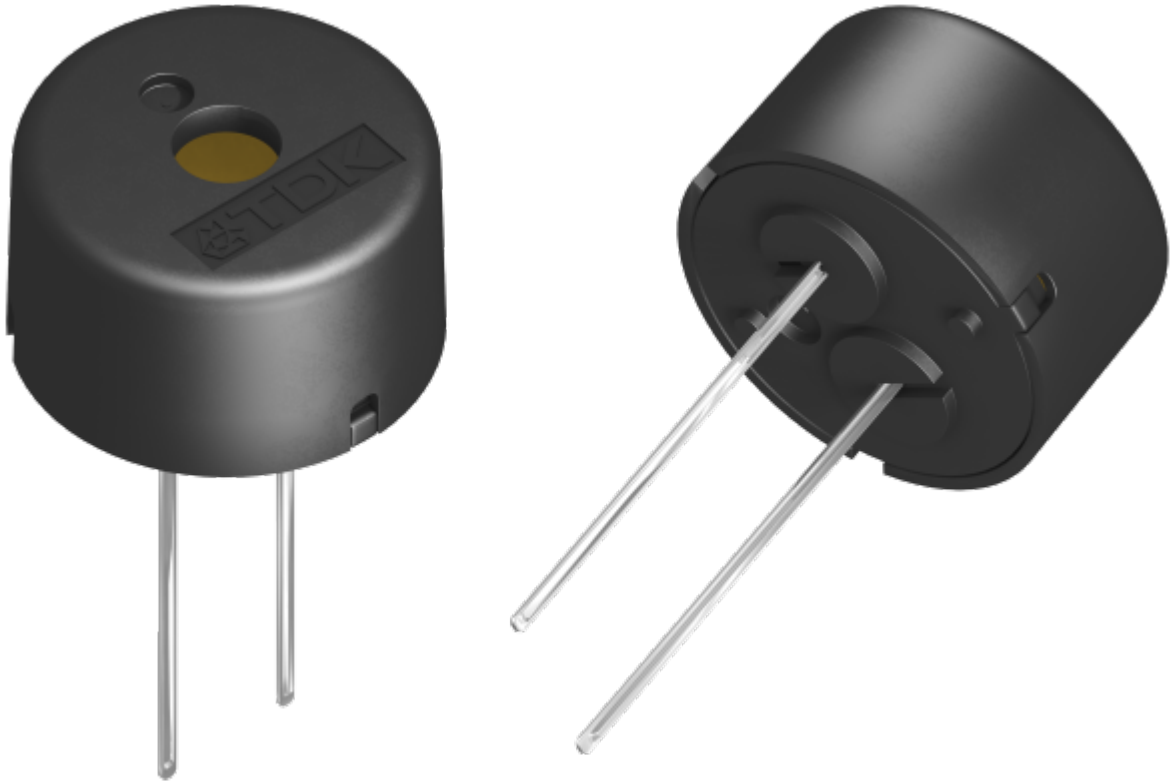
Allows us to have different tones with PWM.

We thus set CMPA = 0, ZRO = 0b11 so that it toggles the output signal and just modify TBPRD.

Since we toggle the output, we need to half the period otherwise we would get the wrong note!

We will go through Fast Fourier Transform next week.

We set the clock divide (divide by 2) so that we can get lower notes!



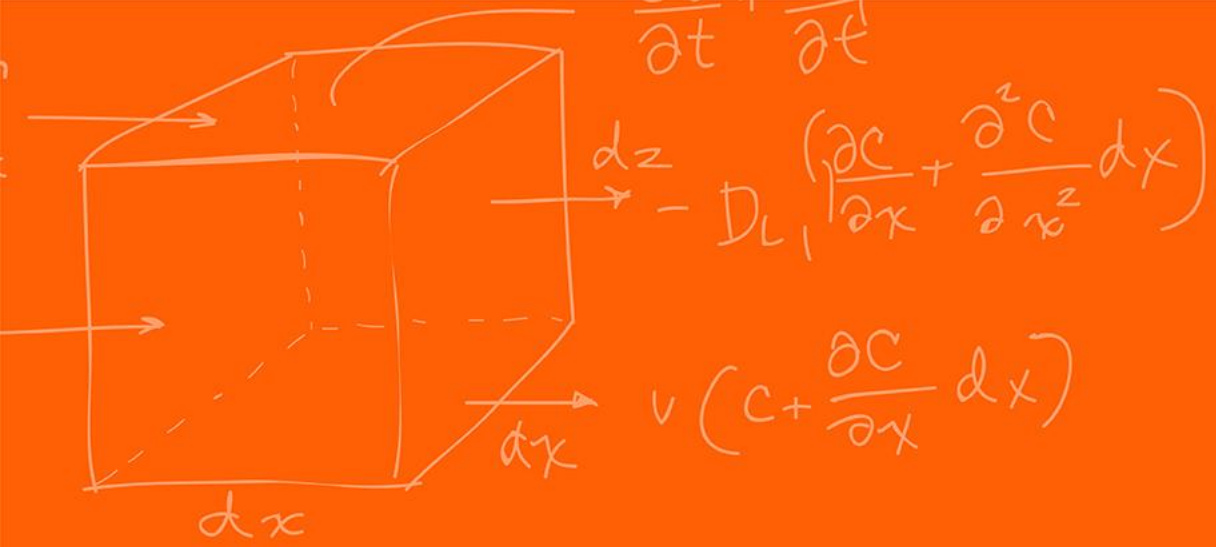
# Piezo Buzzer (2/2)

Divisions handle the clock divide + half frequency

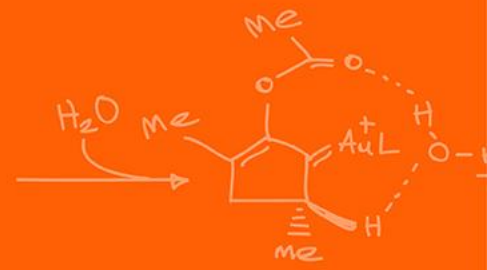
```
#define C4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/261.63)) // TBPRD 47777
#define D4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/293.66)) // TBPRD 42566
#define E4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/329.63)) // TBPRD 37921
#define F4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/349.23)) // TBPRD 35793
#define G4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/392.00)) // TBPRD 31887
#define A4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/440.00)) // TBPRD 28409
#define B4NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/493.88)) // TBPRD 25309
#define C5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/523.25)) // TBPRD 23889
#define D5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/587.33)) // TBPRD 21282
#define E5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/659.25)) // TBPRD 18960
#define F5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/698.46)) // TBPRD 17896
#define G5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/783.99)) // TBPRD 15944
#define A5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/880.00)) // TBPRD 14204
#define B5NOTE        ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/987.77)) // TBPRD 12654
#define F4SHARPNOTE   ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/369.99)) // TBPRD 33784
#define G4SHARPNOTE   ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/415.30)) // TBPRD 30098
#define A4FLATNOTE    ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/415.30)) // TBPRD 30098
#define C5SHARPNOTE   ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/554.37)) // TBPRD 22548
#define A5FLATNOTE    ((uint16_t)(((FUDGEFACTORNOTE*50000000/2)/2)/830.61)) // TBPRD 15049
```

Note frequency! We thus set TBPRD

# Questions? (2/2)









# The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

